

The Title of Project

Hospital Appointment Management System

MajorProject

(23ONMCR753)

of the programme

Masterof ComputerApplications

FourthSemester

CENTREFORDISTANCE& ONLINEEDUCATION

CHANDIGARH UNIVERSITY

CERTIFICATE

Certified that the work contained in the project titled “*Hospital Management System*”, by **Shivani Nene**, has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Declaration

I declare that the work presented in this project titled “*Hospital Management System*”, submitted to the Department of MCA, University of Chandigarh for the award of the **MCA** degree, is my original work. I have not plagiarized or submitted the same work for the award of any other degree. In case this undertaking is found incorrect, I accept that my degree may be unconditionally withdrawn.

April 2026

s.v.nene

(Shivani Nene)

Place: Pune

(Student Name)

Acknowledgement

An important part of a MCA student's life forms the project work that she/he does as it tests the actual practical knowledge gained by the students

that will help them to gain a strong foothold in the largely competitive software industry. As it is, this is very crucial phase for any student and to stand strong the students always require proper and timely guidance.

First of all I would like to thanks Mentor in spite of their busy schedules. We never found any of our requests of help and guidance being turned down by them. Nor did they show the slightest bit of impatience about us pestering them so much and so often.

I would also like to thanks without whose effective guidance. I would have not been able to complete our project in the perfect sense.

I also express my attitude towards my parents and all teachers who extended their cooperating so that I could gain well from that knowledge used in the development of my project.

I would again like to express a heartfelt "Thank You" to everyone involve in this project because without their support this project is not possible.

Shivani Nene
Student Name

Abstract

The need for healthcare services is growing with the increase in population and the number of patients who seek health care at hospitals, medical facilities, holistic groups, and physicians practice has improved significantly.

These bring a new set of challenges for the staff of the facility and administrators. Online scheduling software, a recent technological advancement, has made the booking process in hospitals easier for both patients and administrative staffs.

In case of manual appointment scheduling, there are many disadvantages [1]:

- Constant phone calls that lower productivity: patients, to make appointments and not come to a clinic personally, will be calling the clinic's phone. That might cause quite a number of problems:
 - o There might be missed calls because the line would be occupied at that moment. That also signifies a loss of one customer.
 - o A receptionist can be occupied quite often by phone calls and being unable to attend patients in the lobby waiting. This means that, in order to have a proper service, at least two receptionists would be necessary which might be quite expensive for small clinics.
- Limited by office hours: potential customers that can only come or call to a clinic at one particular time are lost customers if, at that time, a clinic is not working.
- More staff needed: Like we stated before, more personal might be needed (especially receptionists), that means an increase in monthly expenses. Small/medium medical clinics might struggle with such an increase of expenses. Therefore, small-medium clinics can and should have an online website which will permit many different advantages:

- Online appointment scheduling: potential clients (patients) will be able to make appointments online. They won't need to come physically to a clinic or call a phone anymore. The main intention behind designing and developing this project is to enable users to transact in an easy way and it also maintains the details about the available record and appointment details.
- This project aims to develop an efficient and user-friendly system that manages patient information, appointments, billing, medical records, and staff details in a centralized database. The system reduces manual paperwork, minimizes errors, and enhances the overall quality of patient care. It provides secure access to authorized users, ensuring confidentiality and integrity of sensitive data. Features such as online appointment booking, real-time updates, and automated report generation improve operational efficiency and decision-making. The HMS also facilitates better coordination between departments, leading to faster service delivery. By integrating modern technologies, the system supports scalability and adaptability to future needs. Overall, this project demonstrates how digital solutions can optimize hospital workflows, improve resource utilization, and deliver a seamless healthcare experience for both patients and medical professionals.

Contents

Front Page

Certificate

Declaration

Acknowledgement

Abstract

Introduction

SDLC of project

Objectives

Hardware Requirements

Software Requirements

Broad Areas of Applications

Design, Coding & Implementation

Executable files and Output of the project

References

1.Introduction

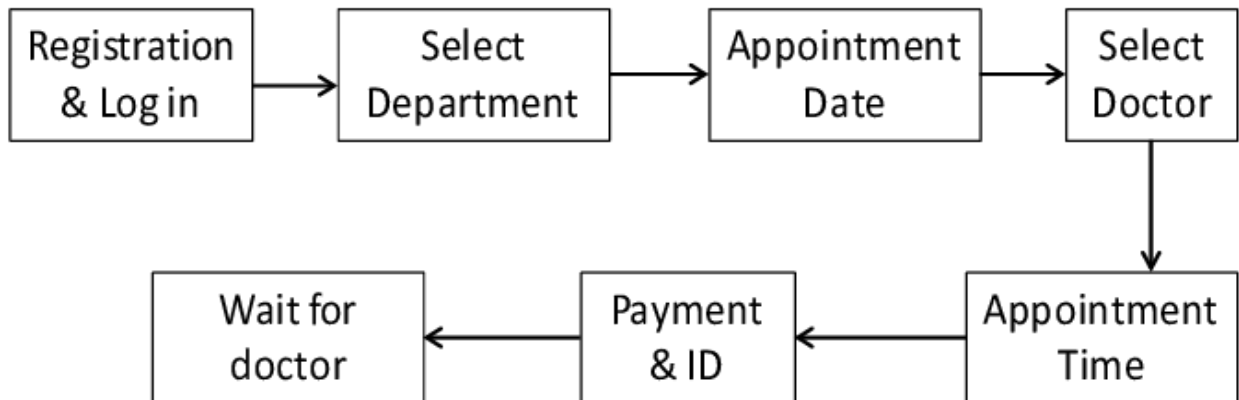
The need for healthcare services is growing with the increase in population and the number of patients who seek health care at hospitals, medical facilities, holistic groups, and physicians practice has improved significantly.

These bring a new set of challenges for the staff of the facility and administrators. Online scheduling software, a recent technological advancement, has made the booking process in hospitals easier for both patients and administrative staffs.

This is designed to improve the efficiency and effectiveness of healthcare services by integrating various hospital operations into a single digital platform. In traditional systems, managing patient records, appointments, billing, and staff information manually can be time-consuming and prone to errors. This project introduces a computerized solution that simplifies these tasks and ensures accurate data management. The HMS enables healthcare professionals to access patient information quickly, schedule appointments efficiently, and maintain medical records securely. It also supports administrative functions such as billing, report generation, and staff management. By reducing

paperwork and automating routine processes, the system enhances productivity and improves patient satisfaction. Additionally, the use of modern technology ensures data security and easy scalability. This project highlights the importance of digital transformation in healthcare and demonstrates how an integrated system can provide reliable, fast, and organized hospital management services.

2. SDLC of the project



3.Objectives

Main objective of hospital management system is:

- 1.Design a system for better patient care.
- 2.Reduce hospital operating cost.
- 3.Provide MIS (Management Information System) report on demand to management for better decision making.
- 4.Better co-ordination among the different departments.
- 5.provide top management a single point of control.
- 6.It aims to reduce waiting times by allowing patients to book appointments in advance, either online or through automated systems, ensuring better time management for both patients and healthcare providers.
- 7.The system helps in minimizing scheduling conflicts, avoiding overbooking, and ensuring optimal utilization of doctors' time and hospital resources.
- 8.It also improves patient satisfaction by providing timely reminders, easy rescheduling options, and quick access to appointment details.

9. Additionally, it maintains accurate digital records of appointments, which supports better coordination among departments and enhances overall operational efficiency.

10. By reducing manual errors and paperwork, the system contributes to improved healthcare service delivery, faster patient flow, and better decision-making through data analysis and reporting.

4. Hardware Requirements:

- 1) Intel Core i3 Processor
- 2) 4GB RAM
- 3) Printer

5. Software Requirements:

- ★ **Front end Tools:**

- ✓ Microsoft Visual Studio 2015

- ★ **RDBMS / Back end Tools**

- ✓ SQL Server 2008

- ★ **Coding Technology**

- ✓ ASP.NET, C#

- ★ **Internet Technologies:**

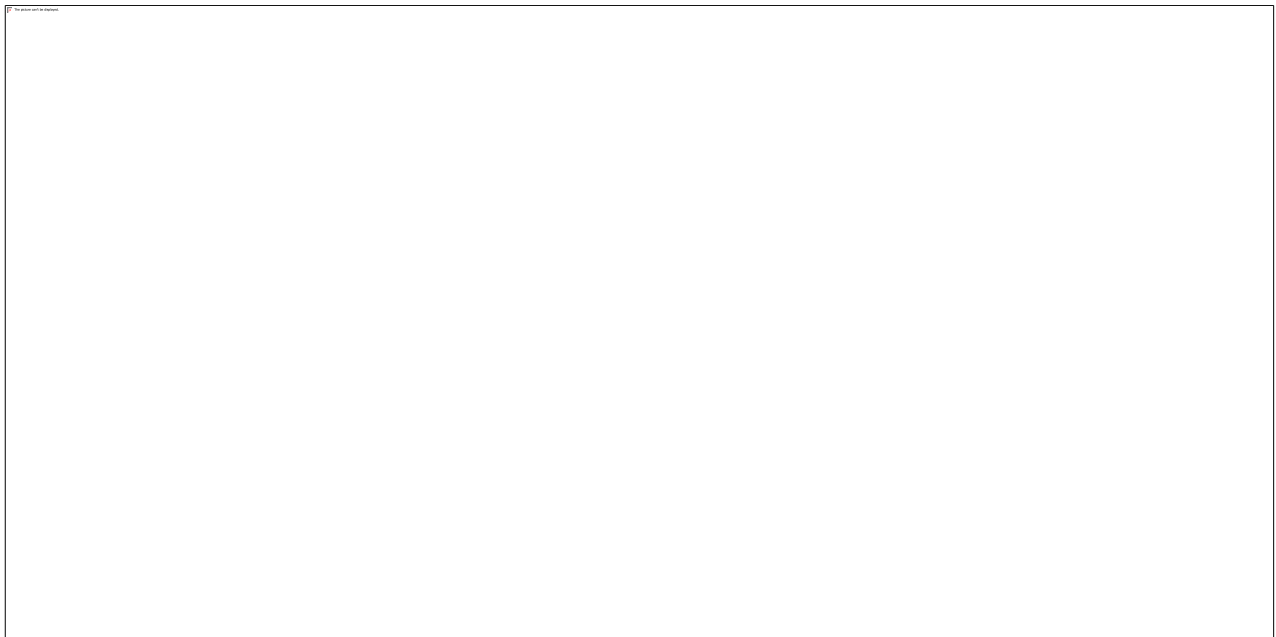
- ✓ TCP/IP, FTP, Telnet, SMTP

- ★ **Operating Systems:**

- ✓ Windows 10

6. Broad Areas of Applications:

- Hospital Management System using C# language.
 - C# (C-sharp) is a modern, object-oriented language created by Microsoft in 2000 as part of the .NET framework. It is used to build Windows applications, web services and more. C# combines the power of C/C++ with the simplicity of Java and Visual Basic.
 -
 - Web Applications & Services: Used to build dynamic websites, RESTful APIs, and web services using frameworks like ASP.NET and ASP.NET Core.
 - Game Development: Widely used in the Unity Engine, one of the most popular platforms for developing 2D, 3D, AR, and VR games.
 - Desktop and Mobile Applications: Supports building Windows desktop applications using WPF and WinForms, as well as cross-platform mobile apps with Xamarin and .NET MAUI.
 - Cloud and Enterprise Solutions: Commonly used for developing scalable cloud applications and enterprise software, especially on platforms like Microsoft Azure.



7. Design, Coding & Implementation

Login.aspx

```
<%@ Page Title="" Language="C#" MasterPageFile="~/MasterPage.master"
AutoEventWireup="true" CodeFile="login.aspx.cs" Inherits="login" %>
```

```
<asp:Content ID="Content1" ContentPlaceHolderID="head" Runat="Server">
```

```
</asp:Content>
```

```
<asp:Content ID="Content2" ContentPlaceHolderID="ContentPlaceHolder1"
Runat="Server">
```

```
<div style="width: 564px; margin-left: 53px; font-size:20px;">
```

```
        <asp:Label ID="Label1" runat="server" Text="Login" as:" Font-
Size="Large" Font-Underline="True" Font-Bold="True" Font-Italic="True"></asp:Label>
```

```
        <br /> <br />
```

```
        <asp:Menu ID="Menu1" runat="server" Orientation="Horizontal"
BackColor="#FFFBD6" DynamicHorizontalOffset="2" Font-Names="Verdana" Font-
Size="0.8em" ForeColor="#990000" StaticSubMenuIndent="10px" >
```

```
            <DynamicHoverStyle BackColor="#990000" ForeColor="White" />
```

```
            <DynamicMenuItemStyle HorizontalPadding="5px" VerticalPadding="2px" />
```

```
            <DynamicMenuStyle BackColor="#FFFBD6" />
```

```
            <DynamicSelectedStyle BackColor="#FFCC66" />
```

```
        </Items>
```

```
            <asp:MenuItem NavigateUrl="~/admin.aspx" Text="admin"
Value="admin"></asp:MenuItem>
```

```
            <asp:MenuItem NavigateUrl="~/patient.aspx" Text="patient"
Value="patient"></asp:MenuItem>
```

```
            <asp:MenuItem NavigateUrl="~/doctor.aspx" Text="doctor"
Value="doctor"></asp:MenuItem>
```

```
        </Items>
```

```
            <StaticHoverStyle BackColor="#990000" ForeColor="White" />
```

```
            <StaticMenuItemStyle HorizontalPadding="5px" VerticalPadding="2px" />
```

```
            <StaticSelectedStyle BackColor="#FFCC66" />
```

```
        </asp:Menu>
```

```
    </div>
```

```
</asp:Content>
```

```
<%@ Page Title="" Language="C#" MasterPageFile="~/MasterPage3.master" AutoEventWireup="true" CodeFile="patient.aspx.cs" Inherits="patient" %>

<asp:Content ID="Content1" ContentPlaceHolderID="head" Runat="Server">
</asp:Content>
<asp:Content ID="Content2" ContentPlaceHolderID="ContentPlaceHolder1" Runat="Server">
    <div style="font-size:30px;">
        please login first.
    </div>
    <div>
        <asp:Label ID="Label1" runat="server" Font-Bold="True" Font-Italic="True" Text="username"></asp:Label>
        &nbsp;
        <asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
        <br />
        <br />
        <asp:Label ID="Label2" runat="server" Font-Bold="True" Font-Italic="True" Text="password"></asp:Label>
        &nbsp;<asp:TextBox ID="TextBox2" runat="server"></asp:TextBox>
        <br />
        <br />
        &nbsp;
        <asp:Label ID="Label3" runat="server" Text=" "></asp:Label>
        <asp:Label ID="Label4" runat="server" Text="Not a member yet!!"></asp:Label>
        &nbsp;<asp:HyperLink ID="HyperLink1" runat="server" ForeColor="#3333FF" NavigateUrl="~/NewLogin.aspx">Register</asp:HyperLink>
        <br />
        <br />
        &nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
        <asp:Button ID="Button1" runat="server" Font-Bold="True" Font-Italic="True" OnClick="Button1_Click" Text="Login" />
    </div></asp:Content>
```

Patient.aspx.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Data;
using System.Data.SqlClient;

public partial class patient : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {

    }

    protected void Button1_Click(object sender, EventArgs e)
    {
        SqlConnection c = new SqlConnection("Data Source=(localdb)\\MSSQLLocalDB;Initial
Catalog=shivaniDB;Integrated Security=True");
        c.Open();

        SqlCommand select = new SqlCommand();
        select.Connection = c;
        int n = Convert.ToInt32(TextBox1.Text);

        select.CommandText = "select userid from patient where userid = '" + n + "' and
password = '" + TextBox2.Text + "' ";
        SqlDataReader reader = select.ExecuteReader();

        if (reader.Read())
        {
            Session["Patient"] = TextBox1.Text;
        }
    }
}
```

```
        Response.Redirect("Patienthome.aspx");  
    }  
    else  
        Label11.Text = "Invalid Login Data";  
    reader.Close();  
    c.Close();  
}  
  
}
```

Appointment.aspx

```
<%@ Page Title="" Language="C#" MasterPageFile="~/MasterPage3.master"
AutoEventWireup="true" CodeFile="appointment.aspx.cs" Inherits="appointment" %>
```

```
<asp:Content ID="Content1" ContentPlaceHolderID="head" Runat="Server">
```

</asp:Content>

```
<asp:Content ID="Content2" ContentPlaceHolderID="ContentPlaceHolder1" Runat="Server">
```

[illegible]

```
<asp:TextBox ID="TextBox4" runat="server"></asp:TextBox>
```

</p>

<p>

```
<asp:Label ID="Label8" runat="server" Font-Bold="True" Font-Italic="True"
Text="date"></asp:Label>
```

```
<asp:Calendar ID="Calendar1" runat="server"></asp:Calendar>
```

</p>

<p>

```
<asp:Label ID="Label9" runat="server" Font-Bold="True" Font-Italic="True"
Text="time"></asp:Label>
```

 :

```
<asp:TextBox ID="TextBox5" runat="server"></asp:TextBox>
```

</p>

<p>

```
    <asp:Label ID="Label7" runat="server" Font-Bold="True" Font-Italic="True"
Text="Dname"></asp:Label>
```

```
<asp:ListBox ID="ListBox1" runat="server">
```

```
<asp:ListItem>dinesh</asp:ListItem>
```

```
<asp:ListItem>kirti</asp:ListItem>
```

```
<asp:ListItem>riya</asp:ListItem>
```

</asp:ListBox>

</p>

<p>

```
<asp:Button ID="Button1" runat="server" Font-Bold="True" Font-Italic="True"
OnClick="Button1_Click" Text="Check" />

    </p>

    <p>
        <asp:GridView ID="GridView1" runat="server">
        </asp:GridView>
        <br />
    </p>

    <p>
        <asp:Label ID="Label10" runat="server" Text="Label"></asp:Label>
    </p>

    <p>
        <br />
    </p>

</asp:Content>
```

Appointment.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Data;
using System.Data.SqlClient;
using System.Diagnostics;

public partial class appointment : System.Web.UI.Page
{
    SqlConnection con;
    SqlCommand cmd;
    SqlDataReader re;

    public void TakeAppointment(int a,int d,int t,string c)
    {
        try
        {
            con = new SqlConnection("Data Source=(localdb)\\MSSQLLocalDB;Initial
Catalog=shivaniDB;Integrated Security=True");
            con.Open();
            int val;
            cmd = new SqlCommand("select status from BookAppointment1 where pid=" +a,
con);

            re = cmd.ExecuteReader();
            GridView1.DataSource = re;
            GridView1.DataBind();
            val =Convert.ToInt32(GridView1.Rows[0].Cells[0].Text);
            if (val == 1)// here 1 means appointment available
```

```
{
    Label10.Text="successfully booked";
}
else if (val == 0) //Here 0 means appointment unavailable
{
    Label10.Text = "not booked";
}

con.Close();
}
catch (Exception es) { Response.Write(es.Message); }
}

protected void Page_Load(object sender, EventArgs e)
{

}

protected void Button1_Click(object sender, EventArgs e)
{
    int a, d,t;
    string c;
    a = Convert.ToInt32(TextBox4.Text);
    d = Convert.ToInt32(Calendar1.SelectedDate.Day);
    t =Convert.ToInt32(TextBox5.Text);
    c = ListBox1.SelectedItem.Value;
    TakeAppointment(a, d, t, c);
}

protected void Timer1_Tick(object sender, EventArgs e)
{

}

}
```


8.Executable files:



WebApplication1_HMS_project.zip

Case Study: Hospital Appointment Management System

A mid-sized hospital was experiencing inefficiencies in managing patient appointments due to its manual scheduling process. Patients had to call or visit the hospital to book appointments, often resulting in long waiting times, double bookings, and frequent delays. Doctors' schedules were poorly managed, leading to uneven workloads and reduced productivity. These issues negatively impacted patient satisfaction and overall hospital operations.

To address these challenges, the hospital implemented a Hospital Appointment Management System. The system allowed patients to book appointments online through a website or mobile application, providing real-time information on doctor availability. It also included automated reminders via SMS and email, reducing missed appointments. Staff could easily manage, reschedule, or cancel bookings, while all appointment data was stored in a centralized digital database.

After implementation, the hospital observed significant improvements. Patient waiting times were reduced, and scheduling conflicts were minimized. Doctors were able to manage their time more effectively, resulting in improved service delivery. Additionally, administrative workload decreased as manual record-keeping was eliminated. Overall, the system enhanced operational efficiency, improved patient experience, and demonstrated how digital solutions can modernize healthcare services and ensure better resource utilization.

Results & Benefits

Waiting time reduced by 40%

Appointment errors significantly minimized

Improved patient satisfaction

Better time utilization for doctors

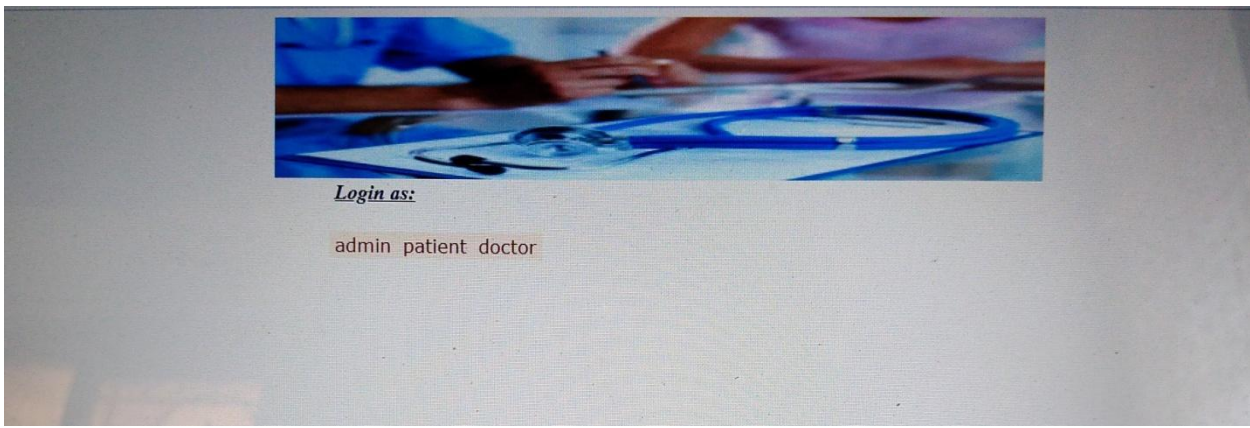
Enhanced data management and reporting

Conclusion

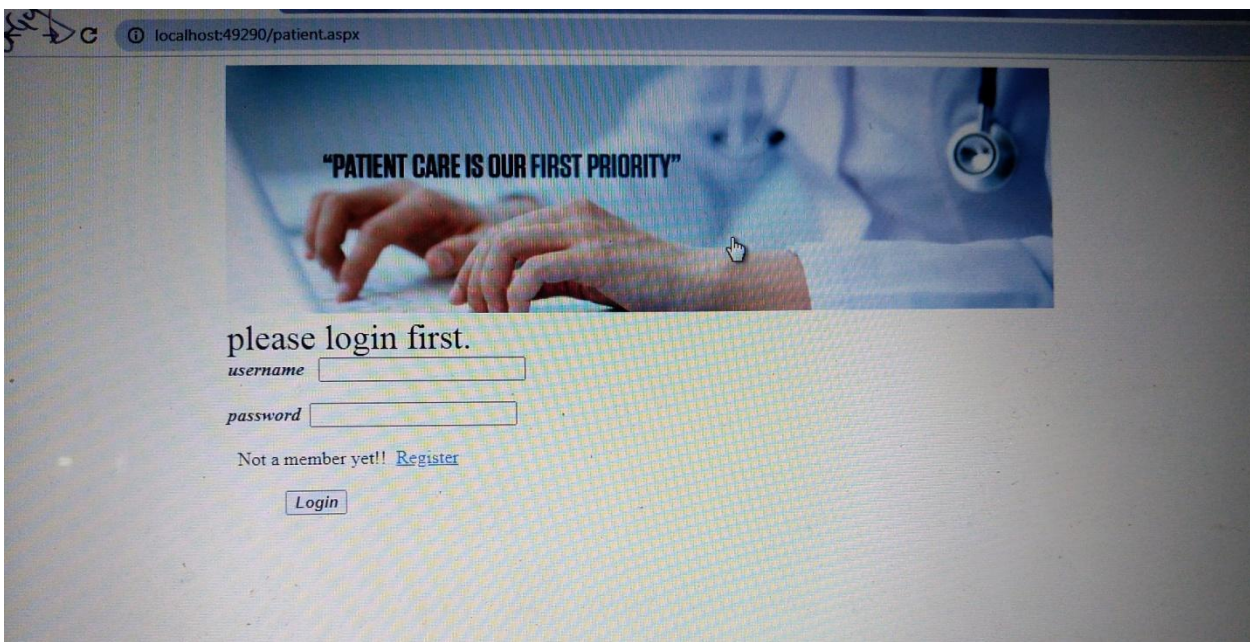
The system improved operational efficiency and patient experience, demonstrating the importance of digital solutions in modern healthcare management.

9. Testing and Output of the code

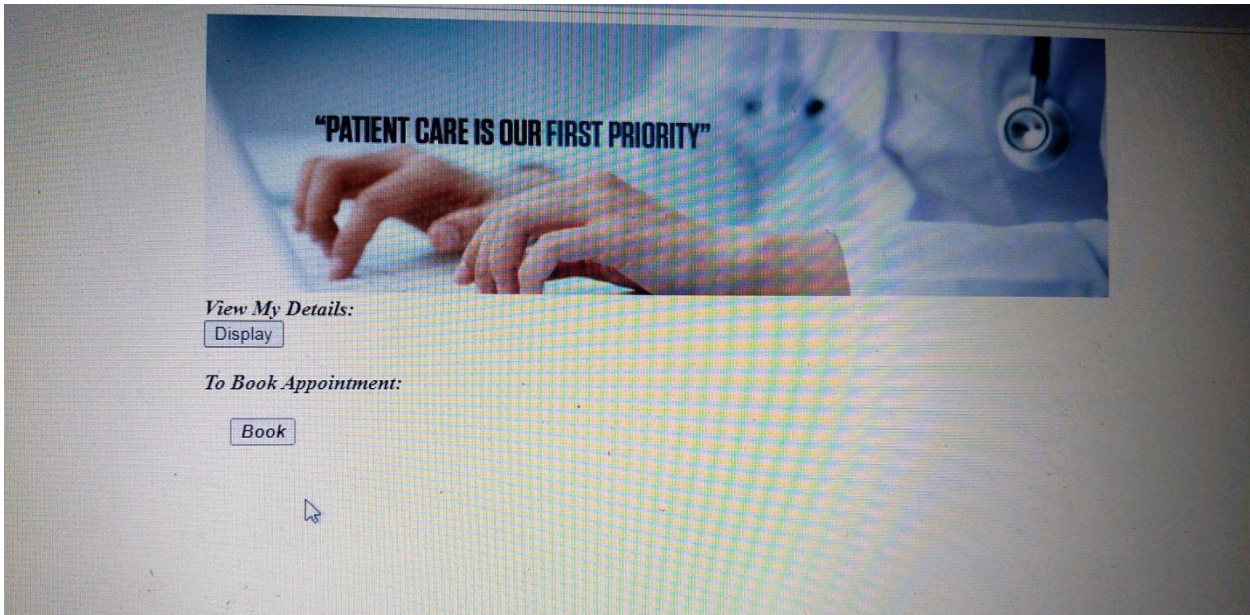
Login.aspx



Patient.aspx



Patient.aspx.cs



Appointment.aspx

Patients id

date :

September 2020						
Sun	Mon	Tue	Wed	Thu	Fri	Sat
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

time :

Dname

status

successfully booked

User Guide for Hospital Appointment Management System (End Users)

A Hospital Appointment Management System is designed to make booking and managing medical appointments simple, fast, and convenient. This guide helps end users (patients) understand how to use the system effectively.

1. Registration and Login

To begin, users must create an account by registering with basic details such as name, phone number, email, and password. After registration, log in using your credentials. Some systems may offer OTP-based login for quick and secure access.

2. Searching for Doctors

Once logged in, users can search for doctors based on specialization (e.g., cardiologist, dermatologist), availability, or department. Filters may also include location, consultation fees, or ratings, helping users choose the right doctor easily.

3. Booking an Appointment

After selecting a doctor, users can view available time slots. Choose a suitable date and time, then confirm the appointment. You may need to provide additional details such as symptoms or reason for visit. A confirmation message will be sent via SMS or email.

4. Viewing Appointments

Users can access a dashboard to view all upcoming and past appointments. This section provides details such as doctor name, appointment time, and hospital location. It helps users stay organized and informed.

5. Rescheduling or Cancelling

If plans change, users can easily reschedule or cancel appointments through the system. Simply select the appointment and choose a new time slot or cancel it. Some systems may have cancellation policies or time limits.

6. Receiving Notifications

The system sends reminders before the appointment to reduce the chances of missing it. Notifications may include appointment confirmations, updates, or changes in schedule.

Conclusion

Medical Appointment application is a web-based mobile application designs to help in patient scheduling with practices in Batu Pahat and Parit Raja area. Based on the developed application, some of the advantages has been identified. The application has gather some medical practices in Batu Pahat and Parit Raja area. User, the patient can easily book their appointment with the practice they wanted avoiding a long queue at the clinic.

During the analysis and development of the Medical Appointment Application, there are some drawbacks that has been identified. The application is unable to allow practices' administrators to show the unavailable appointment time. It only allows user, the patient to fill in the appointment form without knowing which appointment time is available for them.

The constraints that has been faced during the project development is time management and learning to build application using Android platform. Due to improper time management, the developed application is not of a top-notch quality. Besides that, learning to build the application using the Android platform is a difficult task for a first timer with lack of knowledge as a lot needs to be learned. However, these constraints do not hamper the development of this application and the disadvantages can be improved in the future.

Medical Appointment application has the potential to be improved in the future. There are proposals that have been put forward and serve as a guide for improvement of this system. Some of these proposals are to make the practices' administrators able to display the available appointment time for the user to choose and to make an email notification or any sort of notification for appointment rejection. Besides that, it is suggested to make the application interface more interesting and creatives.

In summary, Medical Appointment application has successfully achieved it objectives drawn at the starting point of project planning. The application is built to gather any medical practices that want to participate in the application, specifically practices in Batu Pahat and Parit Raja area. Also, it is to help the patient in avoiding any difficulties in appointment

booking with these practices.

By integrating various functions such as patient registration, appointment scheduling, medical record management, billing, and staff administration into a unified system, the project reduces manual workload and minimizes the chances

of errors. The system enhances efficiency by providing quick access to accurate information, enabling healthcare professionals to make timely and informed decisions.

Furthermore, the HMS improves patient satisfaction by reducing waiting times and ensuring a smooth flow of services. The secure handling of data maintains patient confidentiality and supports compliance with healthcare standards. The system is designed to be user-friendly, scalable, and adaptable to the changing needs of hospitals and clinics.

In conclusion, this project highlights the importance of adopting modern technological solutions in the healthcare sector. It not only improves operational efficiency but also enhances the overall quality of patient care. The implementation of such systems can significantly contribute to better resource management, cost reduction, and improved healthcare delivery, making it an essential tool for modern hospitals.

References

[1] In J. A. Storer and editors. M. Cohn, editors, Proc. 2000 IEEE Data Compression Conference, Los Alamitos, California, 2000. IEEE Computer Society Press.

[2] Calgary corpus. 2000.
<ftp://ftp.cpsc.ucalgary.ca/pub/projects/text.compression.corpus>.

[3] <http://www.cs.wisc.edu/niagara/data/>.

[4] N. Abramson. Information Theory and Coding. McGraw-Hill, 1963.

[5] J. Adiego and P. de la Fuente. Mapping words into codewords on ppm. In Proc. 13th Intl. Symp. On String Processing and Information Retrieval (SPIRE'01), pages 181{192. LNCS 4209., 2006.

[6] **C# in a Nutshell: The Definitive Reference** by Joseph Albahari

Widely considered the definitive reference for the language [11, 12, 27].

Best For: Intermediate to advanced programmers needing a comprehensive map of C# and .NET [13, 44].

Highlights: Deep dives into LINQ, concurrency, and asynchrony without "fluff" or long introductions [11, 13, 50].

[7] [C# in Depth](#) by Jon Skeet

The "gold standard" for understanding how the language has evolved [40].

Best For: Experienced developers who want to master advanced features like

async/await, generics, and pattern matching [5.3, 52].

Highlights: Focuses on the *why* behind language features rather than just the *how*

Introduction about languages used for my project:

ASP.NET:

What is ASP.NET ?

It is a web framework designed and developed by Microsoft. It is used to develop websites, web applications and web services. It provides fantastic integration of HTML, CSS and JavaScript. It was first released in January 2002. It is built on the Common Language Runtime (CLR) and allows programmers to write code using any supported .NET language.

ASP.NET works on top of the HTTP protocol, and uses the HTTP commands and policies to set a browser-to-server bilateral communication and cooperation.

ASP.NET applications are compiled codes, written using the extensible and reusable components or objects present in .Net framework. These codes can use the entire hierarchy of classes in .Net framework.

The ASP.NET application codes can be written in any of the following languages:

- C#
- Visual Basic.Net
- Jscript
- J#

ASP.NET is used to produce interactive, data-driven web applications over the internet. It consists of a large number of controls such as text boxes, buttons, and labels for assembling, configuring, and manipulating code to create HTML pages.

ASP.NET allows developers to create robust and scalable web applications by using programming languages such as C# and VB.NET. It follows a server-side architecture, meaning the application logic runs on the server before the result is sent to the user's browser. This ensures better security, performance, and flexibility. ASP.NET provides various development models, including Web Forms, MVC (Model-View-Controller), and Razor Pages, enabling developers to choose the most suitable approach for their projects.

One of the key advantages of ASP.NET is its integration with the .NET ecosystem. It offers built-in tools for authentication, database connectivity, session management, and caching. Additionally, it supports rapid development through reusable components and extensive libraries. Features like state management, error handling, and debugging tools further enhance developer productivity and application reliability.

Applications of ASP.NET

ASP.NET is widely used across different industries for building a variety of applications:

1. **Enterprise Web Applications**

Large organizations use ASP.NET to develop secure and scalable enterprise-level applications such as customer relationship management (CRM) systems, enterprise resource planning (ERP) systems, and internal business portals.

2. **E-commerce Websites**

ASP.NET is commonly used to create feature-rich e-commerce platforms. It supports secure payment gateways, product catalogs, shopping carts, and user authentication, making it suitable for online stores.

3. **Content Management Systems (CMS)**

Developers use ASP.NET to build CMS platforms that allow users to create, manage, and publish digital content easily. These systems are widely used by businesses and media organizations.

4. **Web APIs and Services**

ASP.NET is ideal for developing RESTful APIs and web services that enable communication between different applications. These APIs are essential for mobile apps, cloud services, and third-party integrations.

5. **Cloud-Based Applications**

With integration into cloud platforms like **Microsoft Azure**, ASP.NET enables developers to build scalable and high-performance cloud applications. It supports microservices architecture and containerization.

6. **Healthcare and Education Systems**

ASP.NET is used to develop systems such as hospital management, appointment scheduling, and e-learning platforms due to its reliability and security features.

Understanding ASP.NET

- ASP.NET executes a web page using the [object-oriented programming approach](#). Every element in asp.net is an object which runs on the server. A .NET Common Language Runtime-compliant compiler compiles a .NET web page into an intermediate language, then a JIT compiler converts the intermediate code to native machine code, and this machine code is eventually run on the processor.
- For an ASP.net developer, it's very important to understand the [ASP.NET page life cycle](#) and ASP.NET application life cycle. It's very much important to know how the request is being processed using IIS and how the web page is being processed and getting served to the user.
- As a developer needs to buy expensive software (called Integrated Development Environments, or IDEs) in order to build an application, Development & software cost is much important factors to consider when making a website or web application. Not only this, ASP.NET applications can be accessed with all of the popular databases, MySQL, MariaDB, Postgres, Microsoft SQL Server, CouchDB, and MongoDB. Low cost, High Speed, and vast language support are among the most significant benefits of using Asp.NET.

The ASP.NET Component Model

The ASP.NET component model provides various building blocks of ASP.NET pages. Basically it is an object model, which describes:

- Server side counterparts of almost all HTML elements or tags, such as <form> and <input>.
- Server controls, which help in developing complex user-interface. For example, the Calendar control or the Gridview control.

ASP.NET is a technology, which works on the .Net framework that contains all web-related functionalities. The .Net framework is made of an object-oriented hierarchy. An ASP.NET web application is made of pages. When a user requests an ASP.NET page, the IIS delegates the processing of the page to the ASP.NET runtime system. The ASP.NET runtime transforms the .aspx page into an instance

of a class, which inherits from the base class page of the .Net framework. Therefore, each ASP.NET page is an object and all its components i.e., the server-side controls are also objects.

Advantages of ASP.NET

Some biggest advantages of using asp.net are :

- Coding time is much less in asp.net framework technology.
- It performs better out of the box.
- Applications built in the asp.net framework are too secure.
- Visual Studio, a framework for asp.net, has a complimentary design and rich toolbox.
- ASP.NET provides continuous monitoring.
- With asp.net, deployment is easier than ever.
- It's easier to write and maintain pages.
- ASP.NET runtime closely manages and monitors all processes.

ASP.NET with Cloud Technology

ASP.NET combined with cloud technology enables developers to build modern, scalable, and high-performance web applications. With the rise of cloud computing, traditional on-premise hosting is increasingly replaced by cloud platforms that offer flexibility, cost efficiency, and global accessibility. ASP.NET, especially **ASP.NET Core**, is designed to integrate seamlessly with cloud environments.

Cloud platforms such as **Microsoft Azure**, **Amazon Web Services**, and **Google Cloud Platform** provide infrastructure and services that support ASP.NET applications. These platforms allow developers to deploy applications without worrying about hardware management, server maintenance, or scalability issues.

One of the major advantages of using ASP.NET with cloud technology is **scalability**. Applications can automatically scale up or down based on user demand, ensuring consistent performance during peak and low traffic periods. Another benefit is **cost efficiency**, as organizations only pay for the resources they use instead of investing in expensive infrastructure.

ASP.NET applications in the cloud also benefit from **high availability and reliability**. Cloud providers offer backup, disaster recovery, and load balancing features, ensuring that applications remain accessible even during failures. Security is also enhanced through built-in features like identity management, encryption, and compliance standards.

Additionally, cloud integration supports **DevOps practices**, enabling continuous integration and continuous deployment (CI/CD). Developers can quickly update and deploy applications using tools integrated with ASP.NET and cloud platforms.

Applications

- Cloud-based enterprise applications
- SaaS (Software as a Service) platforms
- Online booking and management systems
- E-commerce websites
- Web APIs for mobile and IoT applications

ASP.NET and API

ASP.NET is a widely used framework developed by **Microsoft** for building dynamic web applications, services, and APIs. One of its most important capabilities is the development of Web APIs, which allow different software systems to communicate with each other over the internet. APIs (Application Programming Interfaces) act as intermediaries that enable applications to exchange data and functionality securely and efficiently.

In ASP.NET, APIs are commonly built using **ASP.NET Web API** or the modern **ASP.NET Core**. These frameworks help developers create RESTful services that use standard HTTP methods such as GET, POST, PUT, and DELETE. For example, a GET request retrieves data, while a POST request sends new data to the server. This structure makes APIs simple, scalable, and easy to integrate with various clients like web browsers, mobile apps, and desktop software.

ASP.NET APIs typically return data in formats like JSON or XML, with JSON being the most commonly used due to its lightweight and easy-to-parse nature. These APIs are designed to be platform-independent, meaning they can be accessed by applications developed in different programming languages such as Java, Python, or JavaScript.

One of the key advantages of using ASP.NET for API development is its strong support for security. It provides built-in features like authentication and authorization, including integration with identity systems and token-based security such as JWT (JSON Web Tokens). This ensures that only authorized users and applications can access sensitive data.

Another important feature is scalability. ASP.NET APIs can handle a large number of requests efficiently, making them suitable for high-traffic applications. With support for cloud platforms like **Microsoft Azure**, developers can deploy APIs that automatically scale based on demand.

ASP.NET also supports dependency injection, routing, model binding, and middleware, which simplify development and improve code maintainability. Developers can structure applications cleanly using layered architectures, making it easier to update and expand functionality.

Applications of ASP.NET APIs

- Mobile app backends
- E-commerce platforms
- Social media integrations
- Payment gateway services
- Healthcare systems like appointment management
- IoT and cloud-based services

ASP.NET combined with API development plays a crucial role in modern software systems. It enables seamless communication between applications, supports secure data exchange, and allows developers to build scalable and high-performance services. With the evolution of

Event Handling in C# with ASP.NET

Event handling is a core concept in **C#** and plays a vital role in building interactive web applications using **ASP.NET**. It allows a program to respond to user actions such as button clicks, form submissions, or page loading events.

In ASP.NET, event handling is based on the **event-driven programming model**, where controls (like buttons, textboxes, and dropdowns) generate events in response to user interaction. These events are handled by writing specific methods known as event handlers in the code-behind file.

Basic Concept of Event Handling

An event in C# is a message sent by an object to signal that an action has occurred. For example, when a user clicks a button, a “Click” event is triggered. In ASP.NET, developers write event handler methods to define what should happen when the event occurs.

A typical structure includes:

- **Event Source:** The control that raises the event (e.g., Button)
- **Event Handler:** The method that responds to the event
- **Event Arguments:** Data associated with the event

Example of Button Click Event

In an ASP.NET Web Forms application, a button click event can be handled as follows:

```
protected void Button1_Click(object sender, EventArgs e)
{
    Label1.Text = "Button Clicked!";
}
```

Here, when the user clicks the button, the `Button1_Click` method executes and updates the label text.

Page Life Cycle Events

ASP.NET also provides built-in page events such as:

- `Page_Load`: Executes when the page is loaded
- `Page_Init`: Initializes page components
- `Page_PreRender`: Executes before rendering the page

Among these, `Page_Load` is the most commonly used event for initializing data when the page is first accessed.

AutoEventWireup Feature

ASP.NET supports automatic event binding using the `AutoEventWireup` property. When set to true, ASP.NET automatically connects events like `Page_Load` to their handlers without explicit wiring in the code.

Event Handling in ASP.NET MVC/Core

In modern frameworks like **ASP.NET Core**, event handling is less UI-driven compared to Web Forms. Instead, it relies on HTTP request handling through controllers and actions. User interactions are processed via routes and action methods rather than traditional control events.

Advantages of Event Handling

- Improves interactivity of web applications
- Separates UI and logic using code-behind
- Makes applications more responsive and user-friendly
- Enables structured and reusable code

Web Services in ASP.NET

Web services are a fundamental part of modern web development, enabling communication between different applications over the internet. In **ASP.NET**, web services allow developers to build reusable, platform-independent components that can be accessed by various clients such as web apps, mobile apps, and desktop software.

A web service is essentially a software system designed to support interoperable machine-to-machine interaction using standard web protocols like HTTP. It allows different systems, often built using different technologies, to exchange data and perform operations seamlessly.

Types of Web Services in ASP.NET

ASP.NET supports two main types of web services:

1. ASMX Web Services

These are traditional web services built using ASP.NET. They rely on SOAP (Simple Object Access Protocol) for communication. Developers create ASMX services by writing methods in a class and exposing them using attributes like `[WebMethod]`. These services use XML for data exchange and are highly standardized but relatively complex.

2. WCF (Windows Communication Foundation)

Windows Communication Foundation is a more advanced framework for building service-oriented applications. It supports multiple communication protocols such as HTTP, TCP, and named pipes. WCF can be used to create secure, reliable, and high-performance services. It also supports SOAP as well as REST-style communication.

3. ASP.NET Web API

Modern applications primarily use **ASP.NET Web API**, which is designed for building RESTful services. These APIs use HTTP methods like GET, POST, PUT, and DELETE and typically exchange data in JSON format. Web API is lightweight, fast, and ideal for web and mobile applications.

How Web Services Work

Web services operate using a client-server model. The client sends a request to the server hosting the web service. The server processes the request and returns a response. For example, a mobile app may call an ASP.NET Web API to retrieve user data from a database.

Key Features of ASP.NET Web Services

- **Platform Independence:** Can be accessed by any system regardless of programming language

- **Reusability:** Services can be reused across multiple applications
- **Scalability:** Can handle large numbers of requests
- **Security:** Supports authentication and authorization mechanisms
- **Interoperability:** Enables communication between different systems

Advantages

ASP.NET web services provide several benefits, including easy integration with other systems, support for cloud deployment, and efficient data exchange. They are highly suitable for distributed applications where components are located on different servers.

Applications of Web Services

- E-commerce applications for payment processing
- Social media integrations
- Mobile app backends
- Healthcare systems (e.g., patient data sharing)
- Cloud-based services using **Microsoft Azure**

Web services in ASP.NET play a crucial role in building connected and scalable applications. From traditional ASMX services to modern Web APIs, ASP.NET provides multiple approaches to meet different development needs.

ASP.NET Web API is a framework provided by **Microsoft** for building HTTP-based services that can be accessed by various clients such as web browsers, mobile apps, and desktop applications. It follows the REST architectural style and uses standard HTTP methods like GET, POST, PUT, and DELETE to perform operations on data. Web API typically returns data in JSON or XML format, making it lightweight and easy to integrate with different platforms. It is widely used for building scalable and secure backend services, especially for modern web and mobile applications that require fast and efficient data communication.

ASP.NET Core:

ASP.NET Core is a modern, open-source, cross-platform framework developed by Microsoft for building web applications, APIs, and cloud-based services. It is a redesign of the earlier ASP.NET framework, built to be lightweight, modular, and highly performant. Unlike traditional ASP.NET, ASP.NET Core can run on Windows, Linux, and macOS, making it suitable for diverse deployment environments. It is widely used for building scalable enterprise applications, microservices, and RESTful APIs.

ASP.NET Core follows a modular and middleware-based architecture. Middleware components are software units that handle requests and responses in a pipeline. Each middleware can process an incoming HTTP request and either pass it to the next component or return a response immediately. This pipeline approach provides flexibility and control over request handling, allowing developers to customize how requests are processed.

One of the key features of ASP.NET Core is built-in dependency injection (DI). It allows developers to manage object lifetimes and dependencies in a clean and maintainable way. Services are registered in a built-in container and injected into controllers or other services automatically. This improves testability, modularity, and loose coupling in applications.

ASP.NET Core has a flexible hosting model. Applications can run using Kestrel, a lightweight cross-platform web server, often placed behind reverse proxies like IIS or Nginx in production. The framework supports built-in configuration systems using JSON files, environment variables, and command-line arguments. Routing is also an essential feature, mapping incoming URLs to specific controllers and actions in MVC applications or endpoints in minimal APIs.

The benefits of ASP.NET Core include high performance, scalability, and security. It is optimized for modern cloud environments and supports asynchronous programming, which improves application responsiveness. The framework is regularly updated and backed by a large developer community. Overall, ASP.NET Core is a powerful choice for building modern web applications and services.

ASP.NET Core includes several advanced concepts that make it suitable for enterprise development. The middleware pipeline can be extended with custom components for logging, authentication, error handling, and request transformation. Developers can insert middleware at any stage, giving precise control over request flow. This makes debugging and monitoring easier in large applications.

ASP.NET Core supports multiple programming models including MVC, Razor Pages, and Minimal APIs. MVC separates application logic into models, views, and controllers, making code organized and maintainable. Razor Pages simplify page-focused scenarios, while Minimal APIs provide a lightweight way to build HTTP services with minimal code. Security features include authentication, authorization, data protection, and HTTPS enforcement.

Performance is one of the strongest advantages of ASP.NET Core. It is designed to handle high concurrency using asynchronous programming with the async and await model. The framework reduces overhead by being modular, allowing only required components to be included. Kestrel, the built-in web server, is highly optimized for speed and scalability. This makes ASP.NET Core suitable for cloud-native and microservice architectures.

Configuration in ASP.NET Core is highly flexible and environment-based. Developers can easily switch settings between development, staging, and production environments without changing code. Deployment options include self-hosting, Docker containers, and cloud platforms like Azure. The ecosystem includes built-in support for logging, caching, and health checks, which are essential for production-ready applications. This improves observability and reliability.

In summary, ASP.NET Core is a modern framework that unifies performance, flexibility, and scalability for web development. Its modular architecture, middleware pipeline, dependency injection system, and multiple hosting options make it highly adaptable. It is widely adopted in enterprise systems, APIs, and cloud applications due to its efficiency and strong community support. Overall, it represents a significant improvement over traditional ASP.NET and continues to evolve with modern development needs. Developers prefer it because it simplifies building secure, fast, and maintainable applications while supporting cross-platform deployment and cloud-native design principles. It integrates well with modern DevOps practices, continuous integration and deployment pipelines, and container orchestration tools such as Kubernetes, making it suitable for scalable enterprise-grade systems. It also benefits from Microsoft support and long-term updates ensuring stability for enterprise applications over long development cycles and production environments globally adopted framework in software industry today

ASP.NET and Web Forms

ASP.NET web forms contain various web pages and GUI applications such as text box, data grid, label, checkbox, hyperlink, etc. It provides flexibility to web pages at run time as well as design time. It also provides a feature to write code in a separate file from the controls. In ASP.NET, web pages can present information to users in any browser or device.

Features of Web Forms

ASP.NET has multiple features and provides many tools to create and develop web applications. Here are some of the features of web forms:

1. Server Controls

It provides a vast set of server controls. These controls are like objects, and they run when they are requested and rendered to the browser. Some web pages are similar to [HTML](#) elements like text-box, button, checkbox, and hyperlink.

2. Mater Pages

Mater Pages is responsible for the consistent layout of our web applications. It gives a proper appearance and standard to different pages.

3. Routing

URL [routing](#) can be configured to a web application. A request URL is a URL that a user enters in a browser to browse in a specific place.

4. Security

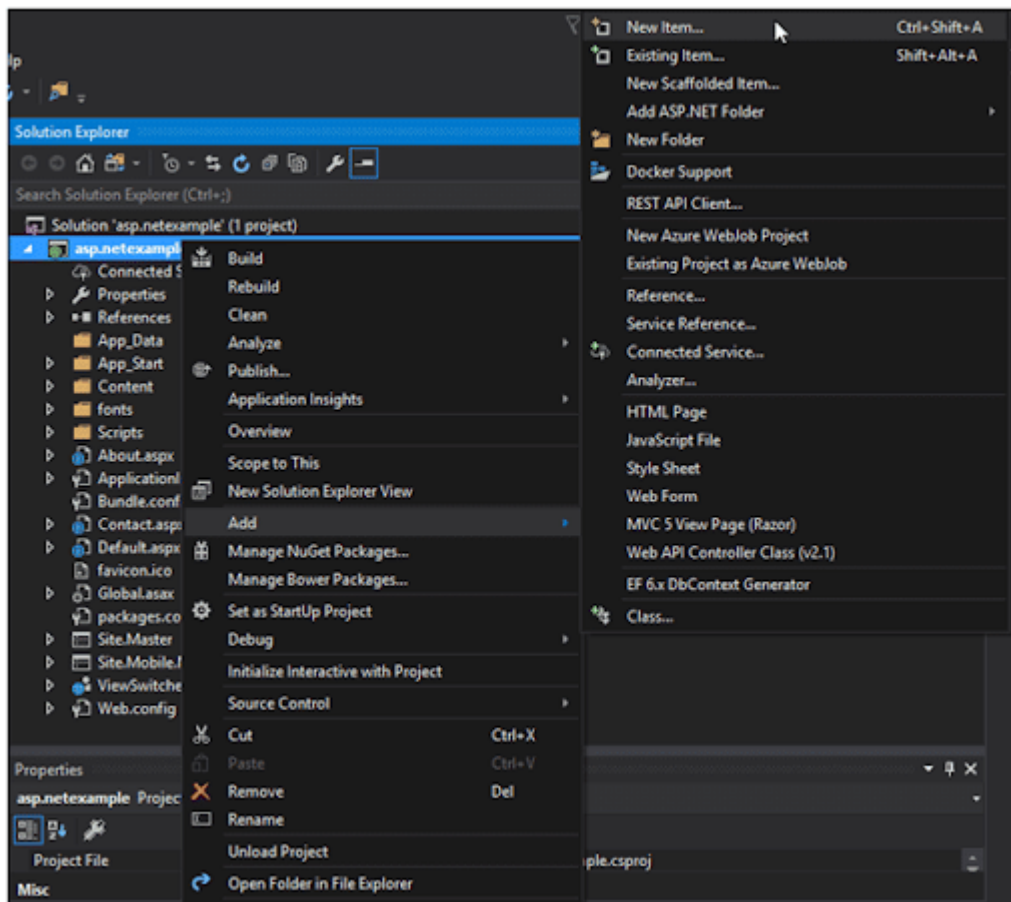
Security always plays a crucial role in software development. ASP.NET provides different configuration options and extensibility points to make our systems more secure.

Example of Web Forms

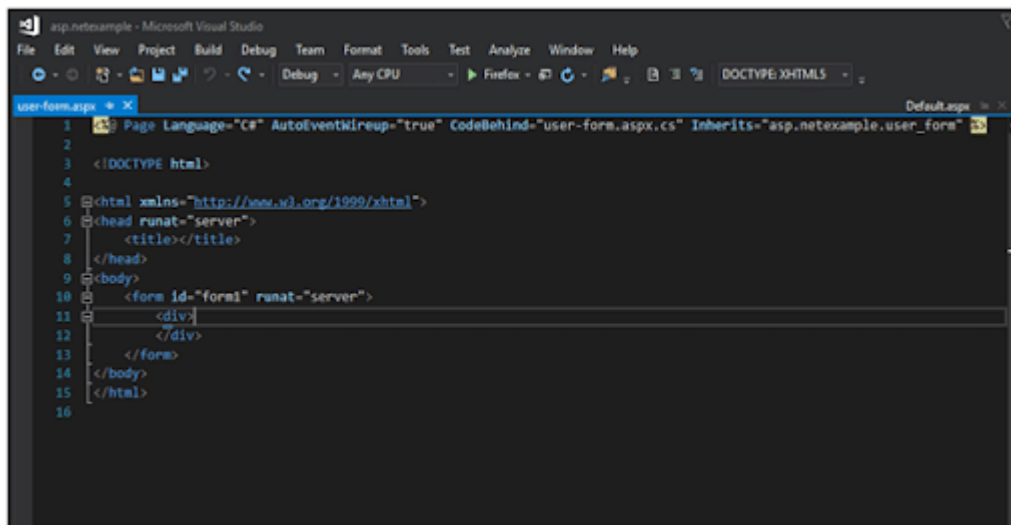
Creating New Web Forms

- To add a new form, select the project, right-click, and add a new item.
- Click the web forms option in the left corner, and select web form.
- Now click on the add button.

This form will add to our project as shown in the below image.



Double click on this form, and this will show some auto-generated code as shown in the image below:



If we run this file on the browser, it does not show any output.

ASP.NET - Life Cycle

ASP.NET life cycle specifies, how:

ASP.NET processes pages to produce dynamic output

The application and its pages are instantiated and processed

ASP.NET compiles the pages dynamically

The ASP.NET life cycle could be divided into two groups:

Application Life Cycle

Page Life Cycle

ASP.NET Application Life Cycle

The application life cycle has the following stages:

User makes a request for accessing application resource, a page. Browser sends this request to the web server.

A unified pipeline receives the first request and the following events take place:

An object of the class ApplicationManager is created.

An object of the class `HostingEnvironment` is created to provide information regarding the resources.

Top level items in the application are compiled.

Response objects are created. The application objects such as `HttpContext`, `HttpRequest` and `HttpResponse` are created and initialized.

An instance of the `HttpApplication` object is created and assigned to the request.

The request is processed by the `HttpApplication` class. Different events are raised by this class for processing the request.

ASP.NET Page Life Cycle

When a page is requested, it is loaded into the server memory, processed, and sent to the browser. Then it is unloaded from the memory. At each of these steps, methods and events are available, which could be overridden according to the need of the application. In other words, you can write your own code to override the default code.

The `Page` class creates a hierarchical tree of all the controls on the page. All the components on the page, except the directives, are part of this control tree. You can see the control tree by adding `trace= "true"` to the page directive. We will cover page directives and tracing under 'directives' and 'event handling'.

The page life cycle phases are:

Initialization

Instantiation of the controls on the page

Restoration and maintenance of the state

Execution of the event handler codes

Page rendering

Understanding the page cycle helps in writing codes for making some specific thing happen at any stage of the page life cycle. It also helps in writing custom controls and initializing them at right time, populate their properties with view-state data and run control behavior code.

Following are the different stages of an ASP.NET page:

Page request - When ASP.NET gets a page request, it decides whether to parse and compile the page, or there would be a cached version of the page; accordingly the response is sent.

Starting of page life cycle - At this stage, the Request and Response objects are set. If the request is an old request or post back, the IsPostBack property of the page is set to true. The UICulture property of the page is also set.

Page initialization - At this stage, the controls on the page are assigned unique ID by setting the UniqueID property and the themes are applied. For a new request, postback data is loaded and the control properties are restored to the view-state values.

Page load - At this stage, control properties are set using the view state and control state values.

Validation - Validate method of the validation control is called and on its successful execution, the IsValid property of the page is set to true.

Postback event handling - If the request is a postback (old request), the related event handler is invoked.

Page rendering - At this stage, view state for the page and all controls are saved. The page calls the Render method for each control and the output of rendering is written to the OutputStream class of the Response property of page.

Unload - The rendered page is sent to the client and page properties, such as Response and Request, are unloaded and all cleanup done.

ASP.NET Page Life Cycle Events

At each stage of the page life cycle, the page raises some events, which could be coded. An event handler is basically a function or subroutine, bound to the event, using declarative attributes such as Onclick or handle.

Following are the page life cycle events:

PreInit - PreInit is the first event in page life cycle. It checks the IsPostBack property and determines whether the page is a postback. It sets the themes and master pages, creates dynamic controls, and gets and sets profile property values. This event can be handled by overloading the OnPreInit method or creating a Page_PreInit handler.

Init - Init event initializes the control property and the control tree is built. This event can be handled by overloading the OnInit method or creating a Page_Init handler.

InitComplete - InitComplete event allows tracking of view state. All the controls turn on view-state tracking.

LoadViewState - LoadViewState event allows loading view state information into the controls.

LoadPostData - During this phase, the contents of all the input fields are defined with the <form> tag are processed.

PreLoad - PreLoad occurs before the post back data is loaded in the controls. This event can be handled by overloading the OnPreLoad method or creating a Page_PreLoad handler.

Load - The Load event is raised for the page first and then recursively for all child controls. The controls in the control tree are created. This event can be handled by overloading the OnLoad method or creating a Page_Load handler.

LoadComplete - The loading process is completed, control event handlers are run, and page validation takes place. This event can be handled by overloading the OnLoadComplete method or creating a Page_LoadComplete handler

PreRender - The PreRender event occurs just before the output is rendered. By handling this event, pages and controls can perform any updates before the output is rendered.

PreRenderComplete - As the PreRender event is recursively fired for all child controls, this event ensures the completion of the pre-rendering phase.

SaveStateComplete - State of control on the page is saved. Personalization, control state and view state information is saved. The HTML markup is generated. This stage can be handled by overriding the Render method or creating a Page_Render handler.

UnLoad - The UnLoad phase is the last phase of the page life cycle. It raises the UnLoad event for all controls recursively and lastly for the page itself. Final cleanup

is done and all resources and references, such as database connections, are freed. This event can be handled by modifying the OnUnload method or creating a Page_UnLoad handler.

WEB Forms in ASP.NET

Various web forms are built using ASP.NET web forms. Web forms are part of the ASP.NET web application and are included in the visual studio. This web form provides us the ability to develop web-based GUI applications. In ASP.NET, you can write reusable code and use it in other places without having to write the code from scratch. Reusability reduces a developer's time, and it also helps in working more efficiently.

ASP.NET web forms contain various web pages and GUI applications such as text box, data grid, label, checkbox, hyperlink, etc. It provides flexibility to web pages at run time as well as design time. It also provides a feature to write code in a separate file from the controls. In ASP.NET, web pages can present information to users in any browser or device. 🌐

Features of Web Forms

ASP.NET has multiple features and provides many tools to create and develop web applications. Here are some of the features of web forms:

1. Server Controls

It provides a vast set of server controls. These controls are like objects, and they run when they are requested and rendered to the browser. Some web pages are similar to [HTML](#) elements like text-box, button, checkbox, and hyperlink.

2. Mater Pages

Mater Pages is responsible for the consistent layout of our web applications. It gives a proper appearance and standard to different pages.

3. Routing

URL [routing](#) can be configured to a web application. A request URL is a URL that a user enters in a browser to browse in a specific place.

4. Security

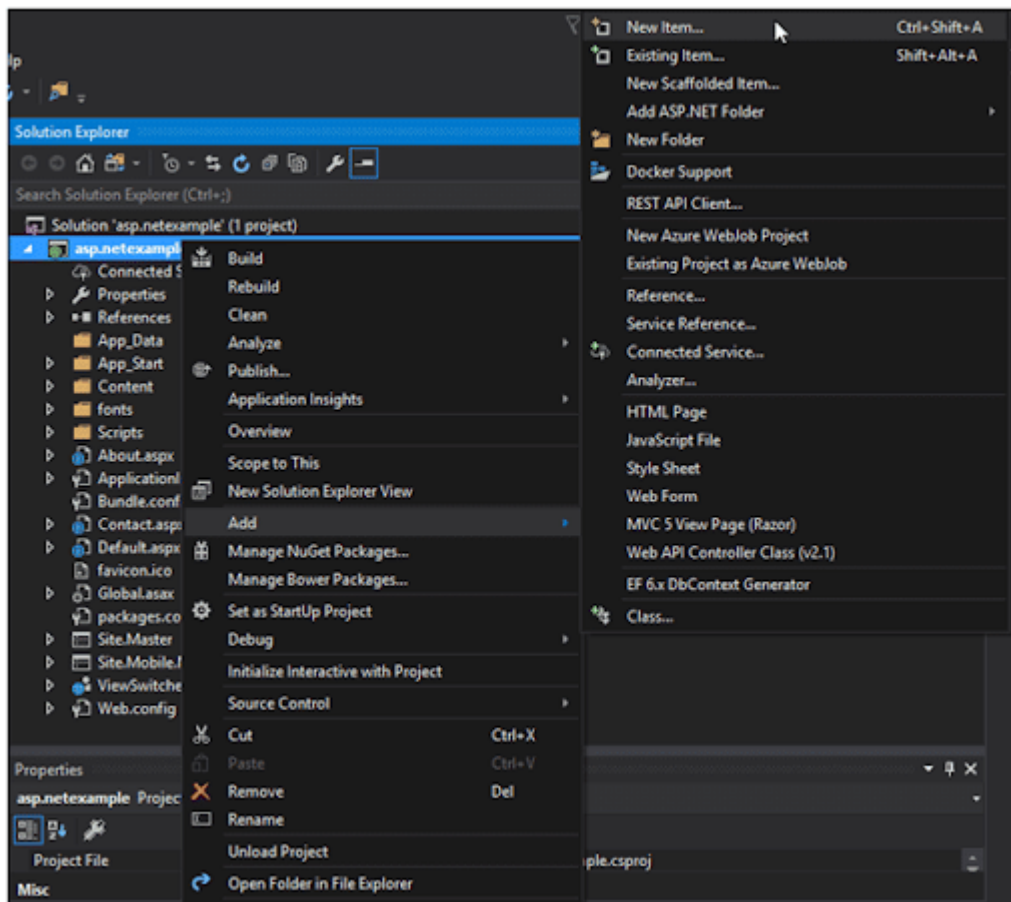
Security always plays a crucial role in software development. ASP.NET provides different configuration options and extensibility points to make our systems more secure.

Example of Web Forms

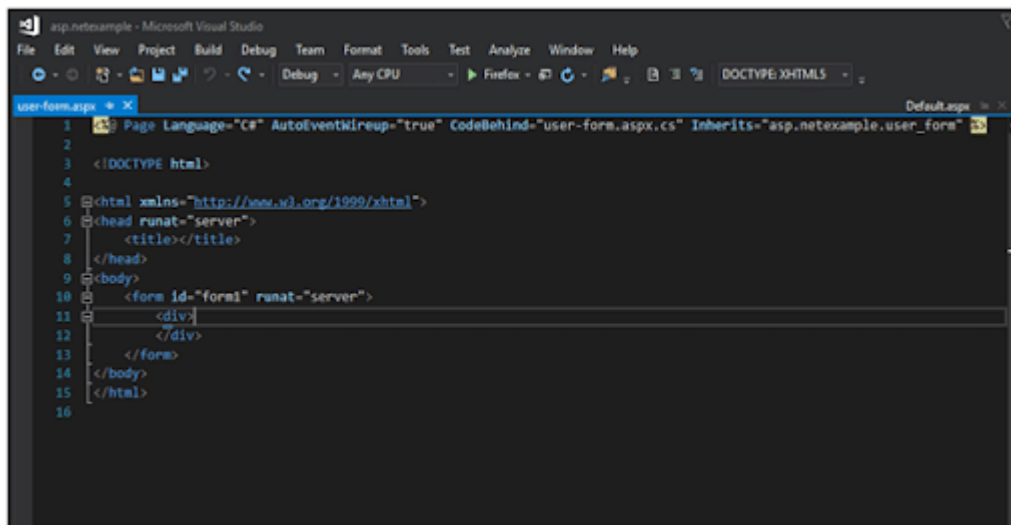
Creating New Web Forms

- To add a new form, select the project, right-click, and add a new item.
- Click the web forms option in the left corner, and select web form.
- Now click on the add button.

This form will add to our project as shown in the below image.



Double click on this form, and this will show some auto-generated code as shown in the image below:



If we run this file on the browser, it does not show any output.

ASP.NET Server Control

ASP.NET has web forms that can be used for making HTML elements. These are of two types: client-based and server-based.

The following table contains some of the server controls:

Control Name	Applicable Events	Description
Label	None	Used for displaying text on an HTML page.
Text Box	Text Changed	Used for creating a text input in the form.
Button	Click, Command	Used for creating a button

ASP.NET Label

This element is mainly responsible for displaying the textual part of the web page. It creates the caption for other controls such as Textbox. We can either write code or drag and drop to create a label.

Example of Label:

```
< asp:LabelID="Label_1" runat="server" Text="Label" ></asp:Label>
```

ASP.NET Text Box

This is a type of input control that is used to take input. It can be created by writing code or by using the drag and drop feature.

As it is a server-side control, ASP provides its own tags.

Example:

```
1. < asp:TextBoxID="Text_Box" runat="server" ></asp:TextBox>
```

ASP.NET Hyperlink

It is a server control used to create a hyperlink and is a click responsive element. A hyperlink is used to refer to any web page through the hyperlink on the server.

ASP.NET gives its own tag for creating it.

ASP.NET Radiobutton

This element mainly takes inputs from the user. It helps the user to select from different options. Radio Button can be created by dragging it from the toolbox of the visual studio.

ASP.NET has its own tag to create it.

ASP.NET Calendar

It provides the feature of displaying the specific date in a calendar. It also helps to show the data associated with that particular date on the web form. This control element enables the user to move to any date, month, and year. Specific dates can also be selected using this feature. We can use the drag and drop function from the toolbar to create it.

ASP.NET Checkbox

It has the feature to take multiple inputs from the user. It allows users to select from a group of choices. It takes input in YES and NO. It plays an important role when we need multiple choices from the user. We can drag it from the toolbox to create it.

ASP.NET has its own tag to create it.

Web Form User Registration

This webform has the crucial function of taking input from the user and submitting it to the server. After submitting it, it also shows that the message has been successfully registered. This whole process gets completed in the following steps:

- Add a web form to the project: This form has some HTML code in it.
- Adding controls to the form: For adding the controls, you can either write your own code or drag and drop it from the toolbox in the visual studio to create components.
- Handling a submit request: In the code file, there is a section where you add a message that only pops out when you submit a registration form.
- Run user registration form: This form takes the inputs from the user form and shows a greeting message to the user.

Web Form Events Handling

ASP.NET provides an essential feature that is event handling to the web forms. It helps to implement an event-based model for your application. In ASP.NET web forms, events associated with server controls mainly originate from clients but are handled on the web server by the ASP.NET. It follows a standard .NET pattern framework for the events handling process.

Various web forms are built using [ASP.NET](#) web forms. Web forms are part of the ASP.NET web application and are included in the visual studio. This web form provides us the ability to develop web-based GUI applications. In ASP.NET, you can write reusable code and use it in other places without having to write the code from scratch. Reusability reduces a developer's time, and it also helps in working more efficiently.

ASP.NET web forms contain various web pages and GUI applications such as text box, data grid, label, checkbox, hyperlink, etc. It provides flexibility to web pages at run time as well as design time. It also provides a feature to write code in a separate file from the controls. In ASP.NET, web pages can present information to users in any browser or device

Features of Web Forms

ASP.NET has multiple features and provides many tools to create and develop web applications. Here are some of the features of web forms:

1. Server Controls

It provides a vast set of server controls. These controls are like objects, and they run when they are requested and rendered to the browser. Some web pages are similar to [HTML](#) elements like text-box, button, checkbox, and hyperlink.

2. Mater Pages

Mater Pages is responsible for the consistent layout of our web applications. It gives a proper appearance and standard to different pages.

3. Routing

URL [routing](#) can be configured to a web application. A request URL is a URL that a user enters in a browser to browse in a specific place.

4. Security

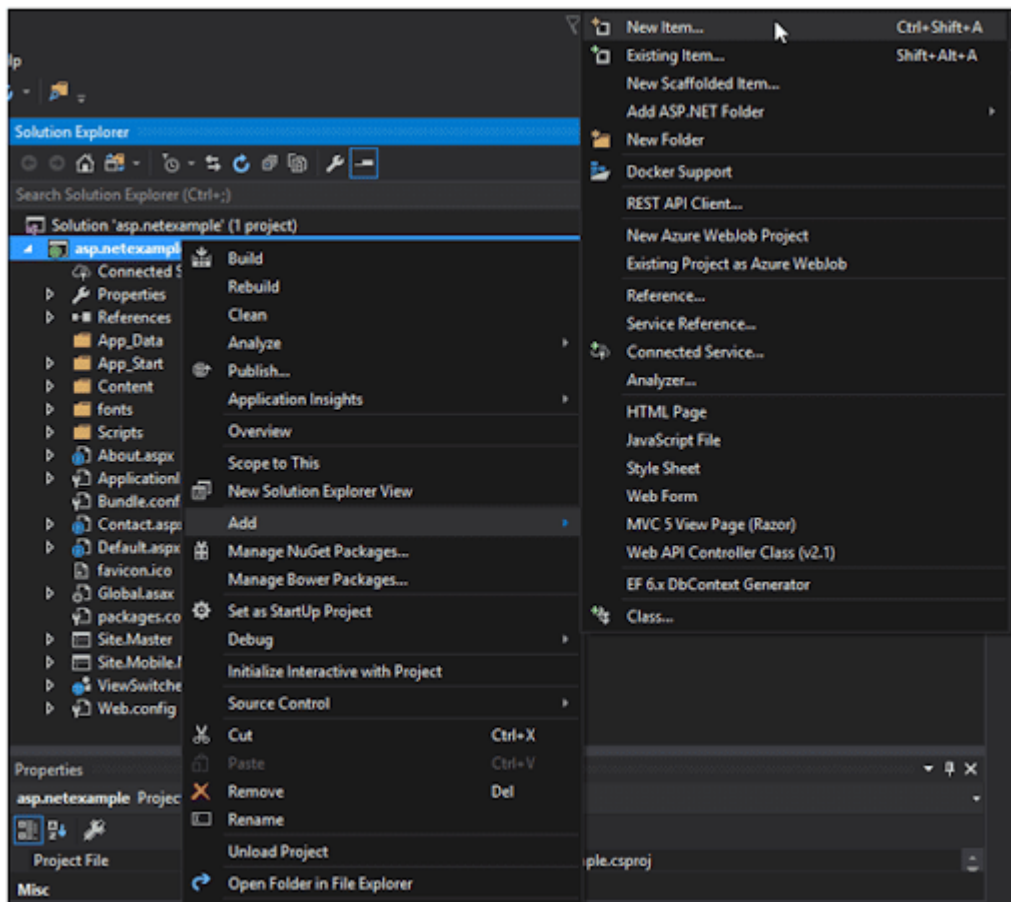
Security always plays a crucial role in software development. ASP.NET provides different configuration options and extensibility points to make our systems more secure.

Example of Web Forms

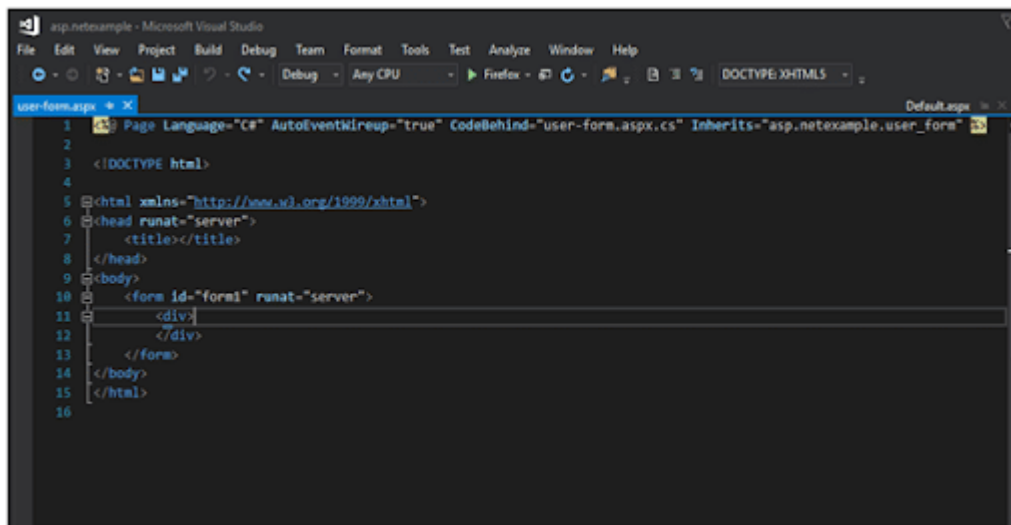
Creating New Web Forms

- To add a new form, select the project, right-click, and add a new item.
- Click the web forms option in the left corner, and select web form.
- Now click on the add button.

This form will add to our project as shown in the below image.



Double click on this form, and this will show some auto-generated code as shown in the image below:



If we run this file on the browser, it does not show any output.

ASP.NET Server Control

ASP.NET has web forms that can be used for making HTML elements. These are of two types: client-based and server-based.

The following table contains some of the server controls:

Control Name	Applicable Events	Description
Label	None	Used for displaying text on an HTML page.
Text Box	Text Changed	Used for creating a text input in the form.
Button	Click, Command	Used for creating a button

ASP.NET Label

This element is mainly responsible for displaying the textual part of the web page. It creates the caption for other controls such as Textbox. We can either write code or drag and drop to create a label.

Example of Label:

```
< asp:LabelID="Label_1" runat="server" Text="Label" ></asp:Label>
```

ASP.NET Text Box

This is a type of input control that is used to take input. It can be created by writing code or by using the drag and drop feature.

As it is a server-side control, ASP provides its own tags.

Example:

```
1. < asp:TextBoxID="Text_Box" runat="server" ></asp:TextBox>
```

ASP.NET Hyperlink

It is a server control used to create a hyperlink and is a click responsive element. A hyperlink is used to refer to any web page through the hyperlink on the server.

ASP.NET gives its own tag for creating it.

ASP.NET Radiobutton

This element mainly takes inputs from the user. It helps the user to select from different options. Radio Button can be created by dragging it from the toolbox of the visual studio.

ASP.NET has its own tag to create it.

ASP.NET Calendar

It provides the feature of displaying the specific date in a calendar. It also helps to show the data associated with that particular date on the web form. This control element enables the user to move to any date, month, and year. Specific dates can also be selected using this feature. We can use the drag and drop function from the toolbar to create it.

ASP.NET provides its own tag to create it.

ASP.NET Checkbox

It has the feature to take multiple inputs from the user. It allows users to select from a group of choices. It takes input in YES and NO. It plays an important role when we need multiple choices from the user. We can drag it from the toolbox to create it.

ASP.NET has its own tag to create it.

Web Form User Registration

This webform has the crucial function of taking input from the user and submitting it to the server. After submitting it, it also shows that the message has been successfully registered. This whole process gets completed in the following steps:

- Add a web form to the project: This form has some HTML code in it.
- Adding controls to the form: For adding the controls, you can either write your own code or drag and drop it from the toolbox in the visual studio to create components.
- Handling a submit request: In the code file, there is a section where you add a message that only pops out when you submit a registration form.
- Run user registration form: This form takes the inputs from the user form and shows a greeting message to the user.

Web Form Events Handling

ASP.NET provides an essential feature that is event handling to the web forms. It helps to implement an event-based model for your application. In ASP.NET web forms, events associated with server controls mainly originate from clients but are handled on the web server by the ASP.NET. It follows a standard .NET pattern framework for the events handling process. 🧑🏻💻

Web Forms Authentication

Web applications have become more safe and secure than before. However, a web network faces various security issues and challenges even today. ASP.NET provides us with the feature of authentications through which we can deal with many problems and filter out the users who can access our application. We can create various types of authentication when creating web applications.

[MVC \(Model-View-Controller\)](#) asks for authentication at the time of creating the web applications. Some of the types of authentication are as follows:

- **No Authentication**

It is used for setting no authentication for the web pages. Any anonymous user can also access this page.

- **Individual User Account**

This type of authentication is most common and popular for web applications. It gives access to an individual user for authentication.

- **Work or School Authentication**

It has the feature of authenticating users with some active directory or a specific domain name. We can also set a particular organization name as authentication.

- **Windows Authentication**

It is mainly responsible for intranet application authentication.

Web Forms Model Binding

Model binding helps to make data interaction more easy and sophisticated. It makes it easy to understand other users. It also does not need to deal with the data source object such as `ObjectDataSource` or `SqlDataSource`.

ADO.NET:

ASP.NET allows the following sources of data to be accessed and used:

- Databases (e.g., Access, SQL Server, Oracle, MySQL)
- XML documents
- Business Objects
- Flat files

ASP.NET hides the complex processes of data access and provides much higher level of classes and objects through which data is accessed easily. These classes hide all complex coding for connection, data retrieving, data querying, and data manipulation.

ADO.NET is the technology that provides the bridge between various ASP.NET control objects and the backend data source. In this tutorial, we will look at data access and working with the data in brief.

Retrieve and display data

It takes two types of data controls to retrieve and display data in ASP.NET:

A data source control - It manages the connection to the data, selection of data, and other jobs such as paging and caching of data etc.

A data view control - It binds and displays the data and allows data manipulation.

ADO.NET provides a bridge between the front end controls and the back end database. The ADO.NET objects encapsulate all the data access operations and the controls interact with these objects to display data, thus hiding the details of movement of data.

The following figure shows the ADO.NET objects at a glance:

ADO.Net objects

The DataSet Class

The dataset represents a subset of the database. It does not have a continuous connection to the database. To update the database a reconnection is required. The DataSet contains DataTable objects and DataRelation objects. The DataRelation objects represent the relationship between two tables.

Following table shows some important properties of the DataSet class:

Properties	Description
CaseSensitive	Indicates whether string comparisons within the data tables are case-sensitive.
Container	Gets the container for the component.
DataSetName	Gets or sets the name of the current data set.
DefaultViewManager	Returns a view of data in the data set.
DesignMode	Indicates whether the component is currently in design mode.
EnforceConstraints	Indicates whether constraint rules are followed when attempting any update operation.
Events	Gets the list of event handlers that are attached to this component.
ExtendedProperties	Gets the collection of customized user information associated with the DataSet.
HasErrors	Indicates if there are any errors.
IsInitialized	Indicates whether the DataSet is initialized.
Locale	Gets or sets the locale information used to compare strings within the table.
Namespace	Gets or sets the namespace of the DataSet.
Prefix	Gets or sets an XML prefix that aliases the namespace of the DataSet.

Relations	Returns the collection of DataRelation objects.
Tables	Returns the collection of DataTable objects.

The DataTable Class

The DataTable class represents the tables in the database. It has the following important properties; most of these properties are read only properties except the PrimaryKey property:

Properties	Description
ChildRelations	Returns the collection of child relationship.
Columns	Returns the Columns collection.
Constraints	Returns the Constraints collection.
DataSet	Returns the parent DataSet.
DefaultView	Returns a view of the table.
ParentRelations	Returns the ParentRelations collection.
PrimaryKey	Gets or sets an array of columns as the primary key for the table.
Rows	Returns the Rows collection.

The DataRow Class

The DataRow object represents a row in a table. It has the following important properties:

Properties	Description
HasErrors	Indicates if there are any errors.
Items	Gets or sets the data stored in a specific column.
ItemArrays	Gets or sets all the values for the row.
Table	Returns the parent table.

The DataAdapter Object

The DataAdapter object acts as a mediator between the DataSet object and the database. This helps the Dataset to contain data from multiple databases or other data source.

The DataReader Object

The DataReader object is an alternative to the DataSet and DataAdapter combination. This object provides a connection oriented access to the data records in the database. These objects are suitable for read-only access, such as populating a list and then breaking the connection.

DbCommand and DbConnection Objects

The DbConnection object represents a connection to the data source. The connection could be shared among different command objects.

The DbCommand object represents the command or a stored procedure sent to the database from retrieving or manipulating data.

Example

So far, we have used tables and databases already existing in our computer. In this example, we will create a table, add column, rows and data into it and display the table using a GridView object.

The source file code is as given:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Default.aspx.cs"
Inherits="createdatabase._Default" %>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml" >
```

```
<head runat="server">
```

```
<title>
```

```
    Untitled Page
```

```
</title>
```

```
</head>
```

```
<body>
```

```
    <form id="form1" runat="server">
```

```
        <div>
```

```
            <asp:GridView ID="GridView1" runat="server">
```

```
                </asp:GridView>
```

```
        </div>
```

```
    </form>
```

```
</body>
```

```
</html>
```

Code backend:

namespace createdatabase

```
{
```

```
    public partial class _Default : System.Web.UI.Page
```

```
    {
```

```
        protected void Page_Load(object sender, EventArgs e)
```

```
        {
```

```
            if (!IsPostBack)
```

```
            {
```

```
                DataSet ds = CreateDataSet();
```

```
                GridView1.DataSource = ds.Tables["Student"];
```

```
                GridView1.DataBind();
```

```
            }
```

```
        }
```

```
private DataSet CreateDataSet()
{
    //creating a DataSet object for tables
    DataSet dataset = new DataSet();

    // creating the student table
    DataTable Students = CreateStudentTable();
    dataset.Tables.Add(Students);
    return dataset;
}

private DataTable CreateStudentTable()
{
    DataTable Students = new DataTable("Student");
    // adding columns
    AddNewColumn(Students, "System.Int32", "StudentID");
    AddNewColumn(Students, "System.String", "StudentName");
    AddNewColumn(Students, "System.String", "StudentCity");
    // adding rows
    AddNewRow(Students, 1, "M H Kabir", "Kolkata");
    AddNewRow(Students, 1, "Shreya Sharma", "Delhi");
    AddNewRow(Students, 1, "Rini Mukherjee", "Hyderabad");
    AddNewRow(Students, 1, "Sunil Dubey", "Bikaner");
    AddNewRow(Students, 1, "Rajat Mishra", "Patna");
    return Students;
}

private void AddNewColumn(DataTable table, string columnType, string columnName)
{
    DataColumn column = table.Columns.Add(columnName, Type.GetType(columnType));
}
```

```
//adding data into the table

private void AddNewRow(DataTable table, int id, string name, string city)
{
    DataRow newrow = table.NewRow();
    newrow["StudentID"] = id;
    newrow["StudentName"] = name;
    newrow["StudentCity"] = city;
    table.Rows.Add(newrow);
}
}
```

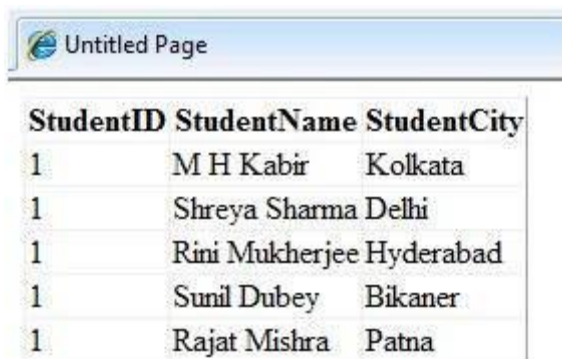
When you execute the program, observe the following:

The application first creates a data set and binds it with the grid view control using the DataBind() method of the GridView control.

The Createdataset() method is a user defined function, which creates a new DataSet object and then calls another user defined method CreateStudentTable() to create the table and add it to the Tables collection of the data set.

The CreateStudentTable() method calls the user defined methods AddNewColumn() and AddNewRow() to create the columns and rows of the table as well as to add data to the rows.

When the page is executed, it returns the rows of the table as shown:



StudentID	StudentName	StudentCity
1	M H Kabir	Kolkata
1	Shreya Sharma	Delhi
1	Rini Mukherjee	Hyderabad
1	Sunil Dubey	Bikaner
1	Rajat Mishra	Patna

ADO.NET and Relational Database System

ADO.NET is a key component of the **.NET Framework** developed by **Microsoft**. It is used for connecting applications to databases and managing data efficiently. ADO.NET provides a set of classes that allow developers to retrieve, insert, update, and delete data from various data sources, especially relational databases.

A **relational database system (RDBMS)** is a type of database that stores data in the form of tables, consisting of rows and columns. Each table represents an entity, and relationships between tables are maintained using keys such as primary keys and foreign keys. Examples of relational database systems include SQL Server, MySQL, and Oracle.

Working of ADO.NET with Relational Databases

ADO.NET acts as a bridge between the application and the relational database. It uses two main components:

1. Connected Architecture

In the connected model, the application remains continuously connected to the database while performing operations. The main component used here is:

- **SqlConnection:** Establishes a connection with the database
- **SqlCommand:** Executes SQL queries or stored procedures
- **SqlDataReader:** Reads data in a fast, forward-only manner

This model is efficient for real-time data access but requires constant database connection.

2. Disconnected Architecture

In this model, the application retrieves data and disconnects from the database. The data is stored temporarily in memory using:

- **DataSet**
- **DataAdapter:** Fills and updates the DataSet

This approach improves scalability and reduces database load since the connection is not continuously maintained.

Features of ADO.NET

- High performance and scalability
- Supports multiple data sources
- Works in both connected and disconnected modes
- XML support for data exchange
- Strong integration with .NET languages like C#

Relational Database System Concepts

A relational database is based on the relational model introduced by E.F. Codd. It organizes data into structured tables. Key concepts include:

- **Table (Relation):** Collection of related data
- **Row (Tuple):** Individual record
- **Column (Attribute):** Field in a table
- **Primary Key:** Uniquely identifies each record
- **Foreign Key:** Establishes relationships between tables

Relational databases use **Structured Query Language (SQL)** for managing data. SQL commands like SELECT, INSERT, UPDATE, and DELETE are commonly used.

Integration of ADO.NET with RDBMS

ADO.NET uses SQL queries to interact with relational databases. For example, a C# application can use `SqlCommand` to fetch student records from a SQL Server database and display them in a web application.

Applications

- Banking systems for transaction management
- E-commerce platforms for product and order management
- Hospital systems for patient and appointment records
- Educational systems for student data management

=====

What is C#?

C# is pronounced "C-Sharp".

It is an object-oriented programming language created by Microsoft that runs on the .NET Framework.

C# has roots from the C family, and the language is close to other popular languages like [C++](#) and [Java](#).

The first version was released in year 2002. The latest version, **C# 13**, was released in November 2024.

C# is used for:

- Mobile applications
- Desktop applications
- Web applications
- Web services
- Web sites
- Games
- VR
- Database applications
- And much, much more!

Why Use C#?

- It is one of the most popular programming languages in the world
- It is easy to learn and simple to use
- It has huge community support
- C# is an object-oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs
- As C# is close to [C](#), [C++](#) and [Java](#), it makes it easy for programmers to switch to C# or vice versa

Basic C# Program

```
using System;
namespace HelloGeeksApp{
    class HelloGeeks{ static void Main(string[] args)
        Console.WriteLine("Hello Geek!"); }}}
```

Core Uses

- **Game Development:** It is the primary language for the [Unity Game Engine](#), used to build millions of 2D, 3D, and VR games.
- **Web Development:** Developers use C# with [ASP.NET](#) to create dynamic websites and secure APIs.
- **Windows Applications:** It is the standard for creating Windows desktop applications using tools like WPF and WinForms.
- **Mobile & Cross-Platform:** Through [.NET MAUI](#) (formerly Xamarin), you can build apps for iOS, Android, and macOS from a single codebase

Key Features

- **Easy to Learn:** It has a clean syntax similar to Java, C++, and JavaScript, making it accessible for beginners.
- **Memory Management:** Unlike C++, C# features [automatic garbage collection](#), which handles memory for you and prevents many common errors.
- **High Performance:** It is a compiled language, offering fast execution times suitable for high-performance applications and games.
- **Strong Community:** It has massive support on platforms like [Stack Overflow](#), with over 5 million developers globally

Language Integrated Query (LINQ)

C# has the ability to utilize [LINQ](#) through the .NET Framework. A developer can query a variety of data sources, provided the `IEnumerable<T>` interface is implemented on the object. This includes [XML](#) documents, an [ADO.NET](#) dataset, and [SQL](#) databases.^[99]

Using [LINQ](#) in C# brings advantages like [IntelliSense](#) support, strong filtering capabilities, type safety with compile error checking ability, and consistency for querying data over a variety of sources.^[100] There are several different language structures that can be utilized with C# and LINQ and they are query expressions, lambda expressions, anonymous types, implicitly typed variables, extension methods, and object initializers.^[101]

LINQ has two syntaxes: query syntax and method syntax. However, the compiler always converts the query syntax to method syntax at compile time.

C# Syntax

In the previous chapter, we created a C# file called Program.cs, and we used the following code to print "Hello World" to the screen:

Program.cs

```
using System;

namespace HelloWorld
{
    class Program
    {
        static void Main(string[] args)
```

```
{  
    Console.WriteLine("Hello World!");  
}  
}
```

Result:

Hello World!

C# Methods

A **method** is a block of code which only runs when it is called.

You can pass data, known as parameters, into a method.

Methods are used to perform certain actions, and they are also known as **functions**.

Why use methods? To reuse code: define the code once, and use it many times.

Create a Method

A method is defined with the name of the method, followed by parentheses (). C# provides some pre-defined methods, which you already are familiar with, such as `Main()`, but you can also create your own methods to perform certain actions:

Example [Get your own C# Server](#)

Create a method inside the Program class:

```
class Program  
{
```

```
static void MyMethod()  
{  
    // code to be executed  
}  
}
```

Call a Method

To call (execute) a method, write the method's name followed by two parentheses **()** and a semicolon;

In the following example, **MyMethod()** is used to print a text (the action), when it is called:

Example

Inside **Main()**, call the **myMethod()** method:

```
static void MyMethod()  
{  
    Console.WriteLine("I just got executed!");  
}  
  
static void Main(string[] args)  
{  
    MyMethod();  
}  
  
// Outputs "I just got executed!"
```

Method Overloading

With **method overloading**, multiple methods can have the same name with different parameters:

Example [Get your own C# Server](#)

```
int MyMethod(int x)

float MyMethod(float x)

double MyMethod(double x, double y)
```

Consider the following example, which has two methods that add numbers of different types:

Example

```
static int PlusMethodInt(int x, int y)
{
    return x + y;
}

static double PlusMethodDouble(double x, double y)
{
    return x + y;
}

static void Main(string[] args)
{
    int myNum1 = PlusMethodInt(8, 5);

    double myNum2 = PlusMethodDouble(4.3, 6.26);
}
```

```
Console.WriteLine("Int: " + myNum1);  
  
Console.WriteLine("Double: " + myNum2);  
  
}
```

Instead of defining two methods that should do the same thing, it is better to overload one.

In the example below, we overload the **PlusMethod** method to work for both **int** and **double**:

Example

```
static int PlusMethod(int x, int y)  
{  
    return x + y;  
}  
  
static double PlusMethod(double x, double y)  
{  
    return x + y;  
}  
  
static void Main(string[] args)  
{  
    int myNum1 = PlusMethod(8, 5);  
    double myNum2 = PlusMethod(4.3, 6.26);  
    Console.WriteLine("Int: " + myNum1);  
    Console.WriteLine("Double: " + myNum2);  
}
```


C# - What is OOP?

OOP stands for Object-Oriented Programming.

Procedural programming is about writing procedures or methods that perform operations on the data, while object-oriented programming is about creating objects that contain both data and methods.

Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C# code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

Tip: The "Don't Repeat Yourself" (DRY) principle is about reducing the repetition of code. You should extract out the codes that are common for the application, and place them at a single place and reuse them instead of repeating it.

C# - What are Classes and Objects?

Classes and objects are the two main aspects of object-oriented programming.

Everything in C# is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an object. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.

A Class is like an object constructor, or a "blueprint" for creating objects.

C# Data Types

As explained in the variables chapter, a variable in C# must be a specified data type:

Example [Get your own C# Server](#)

```
int myNum = 5;           // Integer (whole number)
double myDoubleNum = 5.99D; // Floating point number
char myLetter = 'D';      // Character
bool myBool = true;       // Boolean
string myText = "Hello";  // String
```

A data type specifies the size and type of variable values.

It is important to use the correct data type for the corresponding variable; to avoid errors, to save time and memory, but it will also make your code more maintainable and readable. The most common data types are:

Data Type	Size	Description
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 decimal places

<code>double</code>	8 bytes	Stores fractional numbers. Sufficient for storing 1
<code>bool</code>	1 byte	Stores true or false values
<code>char</code>	2 bytes	Stores a single character/letter, surrounded by sin
<code>string</code>	2 bytes per character	Stores a sequence of characters, surrounded by do

Numbers

Number types are divided into two groups:

Integer types stores whole numbers, positive or negative (such as 123 or -456), without decimals. Valid types are `int` and `long`. Which type you should use, depends on the numeric value.

Floating point types represents numbers with a fractional part, containing one or more decimals. Valid types are `float` and `double`.

Even though there are many numeric types in C#, the most used for numbers are `int` (for whole numbers) and `double` (for floating point numbers). However, we will describe them all as you continue to read.

Integer Types

Int

The `int` data type can store whole numbers from -2147483648 to 2147483647. In general, and in our tutorial, the `int` data type is the preferred data type when we create variables with a numeric value.

Example

```
int myNum = 100000;  
Console.WriteLine(myNum);
```

Long

The `long` data type can store whole numbers from -9223372036854775808 to 9223372036854775807. This is used when `int` is not large enough to store the value. Note that you should end the value with an "L":

Example

```
long myNum = 15000000000L;  
Console.WriteLine(myNum);
```

Floating Point Types

You should use a floating point type whenever you need a number with a decimal, such as 9.99 or 3.14515.

The `float` and `double` data types can store fractional numbers. Note that you should end the value with an "F" for floats and "D" for doubles:

Float Example

```
float myNum = 5.75F;
```

```
Console.WriteLine(myNum);
```

Double Example

```
double myNum = 19.99D;
```

```
Console.WriteLine(myNum);
```

Use **float** or **double**?

The **precision** of a floating point value indicates how many digits the value can have after the decimal point. The precision of **float** is only six or seven decimal digits, while **double** variables have a precision of about 15 digits. Therefore it is safer to use **double** for most calculations.

Scientific Numbers

A floating point number can also be a scientific number with an "e" to indicate the power of 10:

Example

```
float f1 = 35e3F;
```

C# Strings

Strings are used for storing text.

A **string** variable contains a collection of characters surrounded by double quotes:

Example [Get your own C# Server](#)

Create a variable of type **string** and assign it a value:

```
string greeting = "Hello";
```

A string variable can contain many words, if you want:

Example

```
string greeting2 = "Nice to meet you!";
```

String Length

A string in C# is actually an object, which contain properties and methods that can perform certain operations on strings. For example, the length of a string can be found with the `Length` property:

Example

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  
Console.WriteLine("The length of the txt string is: " + txt.Length);
```

[REMOVE ADS](#)

Other Methods

There are many string methods available, for example `ToUpper()` and `ToLower()`, which returns a copy of the string converted to uppercase or lowercase:

Example

```
string txt = "Hello World";  
Console.WriteLine(txt.ToUpper());    // Outputs "HELLO WORLD"  
Console.WriteLine(txt.ToLower());    // Outputs "hello world"
```

C# Strings

Strings are used for storing text.

A `string` variable contains a collection of characters surrounded by double quotes:

Example [Get your own C# Server](#)

Create a variable of type `string` and assign it a value:

```
string greeting = "Hello";
```

A string variable can contain many words, if you want:

Example

```
string greeting2 = "Nice to meet you!";
```

String Length

A string in C# is actually an object, which contain properties and methods that can perform certain operations on strings. For example, the length of a string can be found with the `Length` property:

Example

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
```

```
Console.WriteLine("The length of the txt string is: " + txt.Length);
```

[REMOVE ADS](#)

Other Methods

There are many string methods available, for example `ToUpper()` and `ToLower()`, which returns a copy of the string converted to uppercase or lowercase:

Example

```
string txt = "Hello World";  
Console.WriteLine(txt.ToUpper()); // Outputs "HELLO WORLD"  
Console.WriteLine(txt.ToLower()); // Outputs "hello world"
```

Create an Object

An object is created from a class. We have already created the class named **Car**, so now we can use this to create objects.

To create an object of **Car**, specify the class name, followed by the object name, and use the keyword **new**:

Example [Get your own C# Server](#)

Create an object called "myObj" and use it to print the value of **color**:

```
class Car  
{  
    string color = "red";  
  
    static void Main(string[] args)  
    {  
        Car myObj = new Car();  
        Console.WriteLine(myObj.color);  
    }  
}
```


}

C# Constructors

Constructors

A constructor is a **special method** that is used to initialize objects. The advantage of a constructor, is that it is called when an object of a class is created. It can be used to set initial values for fields:

Example [Get your own C# Server](#)

Create a constructor:

```
// Create a Car class
class Car
{
    public string model; // Create a field

    // Create a class constructor for the Car class
    public Car()
    {
        model = "Mustang"; // Set the initial value for model
    }

    static void Main(string[] args)
    {
        Car Ford = new Car(); // Create an object of the Car Class (this
                               // will call the constructor)
```

```
Console.WriteLine(Ford.model); // Print the value of model  
  
}  
  
}  
  
// Outputs "Mustang"
```

Note that the constructor name must **match the class name**, and it cannot have a **return type** (like `void` or `int`).

Also note that the constructor is called when the object is created.

All classes have constructors by default: if you do not create a class constructor yourself, C# creates one for you. However, then you are not able to set initial values for fields.

Access Modifiers

By now, you are quite familiar with the `public` keyword that appears in many of our examples:

```
public string color;
```

The `public` keyword is an **access modifier**, which is used to set the access level/visibility for classes, fields, methods and properties.

C# has the following access modifiers:

Modifier	Description
<code>public</code>	The code is accessible for all classes

`private`

The code is only accessible within the same class

`protected`

The code is accessible within the same class, or in a class that is inheriting about [inheritance](#) in a later chapter

`internal`

The code is only accessible within its own assembly, but not from another assembly in a later chapter

There's also two combinations: `protected internal` and `private protected`.

For now, let's focus on `public` and `private` modifiers.

Private Modifier

If you declare a field with a `private` access modifier, it can only be accessed within the same class:

Example [Get your own C# Server](#)

```
class Car
{
    private string model = "Mustang";

    static void Main(string[] args)
    {
        Car myObj = new Car();
        Console.WriteLine(myObj.model);
    }
}
```

```
}
```

```
}
```

The output will be:

```
Mustang
```

Properties and Encapsulation

Before we start to explain properties, you should have a basic understanding of "**Encapsulation**".

The meaning of **Encapsulation**, is to make sure that "sensitive" data is hidden from users. To achieve this, you must:

- declare fields/variables as **private**
- provide **public get** and **set** methods, through **properties**, to access and update the value of a **private** field

Properties

You learned from the previous chapter that **private** variables can only be accessed within the same class (an outside class has no access to it). However, sometimes we need to access them - and it can be done with properties.

A property is like a combination of a variable and a method, and it has two methods: a **get** and a **set** method:

Example [Get your own C# Server](#)

```
class Person
{
    private string name; // field

    public string Name    // property
    {
        get { return name; }    // get method
        set { name = value; }    // set method
    }
}
```

Example explained

The **Name** property is associated with the **name** field. It is a good practice to use the same name for both the property and the private field, but with an uppercase first letter.

The **get** method returns the value of the variable **name**.

The **set** method assigns a **value** to the **name** variable. The **value** keyword represents the value we assign to the property.

C# Inheritance

Inheritance (Derived and Base Class)

In C#, it is possible to inherit fields and methods from one class to another. We group the "inheritance concept" into two categories:

- **Derived Class** (child) - the class that inherits from another class
- **Base Class** (parent) - the class being inherited from

To inherit from a class, use the `:` symbol.

In the example below, the `Car` class (child) inherits the fields and methods from the `Vehicle` class (parent):

Example [Get your own C# Server](#)

```
class Vehicle // base class (parent)
{
    public string brand = "Ford"; // Vehicle field
    public void honk()           // Vehicle method
    {
        Console.WriteLine("Tuut, tuut!");
    }
}

class Car : Vehicle // derived class (child)
```

```
{  
  
    public string modelName = "Mustang"; // Car field  
  
}  
  
class Program  
{  
  
    static void Main(string[] args)  
    {  
  
        // Create a myCar object  
  
        Car myCar = new Car();  
  
  
        // Call the honk() method (From the Vehicle class) on the myCar  
object  
  
        myCar.honk();  
  
  
        // Display the value of the brand field (from the Vehicle class) and  
the value of the modelName from the Car class  
  
        Console.WriteLine(myCar.brand + " " + myCar.modelName);  
  
    }  
  
}
```

C# Polymorphism

Polymorphism and Overriding Methods

Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance.

Like we specified in the previous chapter; [Inheritance](#) lets us inherit fields and methods from another class. **Polymorphism** uses those methods to perform different tasks. This allows us to perform a single action in different ways.

For example, think of a base class called `Animal` that has a method called `animalSound()`. Derived classes of Animals could be Pigs, Cats, Dogs, Birds - And they also have their own implementation of an animal sound (the pig oinks, and the cat meows, etc.):

Example [Get your own C# Server](#)

```
class Animal // Base class (parent)
{
    public void animalSound()
    {
        Console.WriteLine("The animal makes a sound");
    }
}

class Pig : Animal // Derived class (child)
{
    public void animalSound()
```



```
{  
    Console.WriteLine("The pig says: wee wee");  
}  
}  
  
class Dog : Animal // Derived class (child)  
{  
    public void animalSound()  
    {  
        Console.WriteLine("The dog says: bow wow");  
    }  
}
```

Now we can create **Pig** and **Dog** objects and call the **animalSound()** method on both of them:

Example

```
class Animal // Base class (parent)  
{  
    public void animalSound()  
    {  
        Console.WriteLine("The animal makes a sound");  
    }  
}  
  
class Pig : Animal // Derived class (child)  
{
```

```
public void animalSound()
{
    Console.WriteLine("The pig says: wee wee");
}
}

class Dog : Animal // Derived class (child)
{
    public void animalSound()
    {
        Console.WriteLine("The dog says: bow wow");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Animal myAnimal = new Animal(); // Create a Animal object
        Animal myPig = new Pig(); // Create a Pig object
        Animal myDog = new Dog(); // Create a Dog object

        myAnimal.animalSound();
        myPig.animalSound();
        myDog.animalSound();
    }
}
```

```
}  
  
}
```

The output will be:

```
The          animal          makes          a          sound  
The          animal          makes          a          sound  
The animal makes a sound
```

However, C# provides an option to override the base class method, by adding the **virtual** keyword to the method inside the base class, and by using the **override** keyword for each derived class methods:

Example

```
class Animal // Base class (parent)
{
    public virtual void animalSound()
    {
        Console.WriteLine("The animal makes a sound");
    }
}

class Pig : Animal // Derived class (child)
{
    public override void animalSound()
    {
        Console.WriteLine("The pig says: wee wee");
    }
}
```

```
}  
  
}  
  
class Dog : Animal // Derived class (child)  
{  
    public override void animalSound()  
    {  
        Console.WriteLine("The dog says: bow wow");  
    }  
}  
  
class Program  
{  
    static void Main(string[] args)  
    {  
        Animal myAnimal = new Animal(); // Create a Animal object  
        Animal myPig = new Pig(); // Create a Pig object  
        Animal myDog = new Dog(); // Create a Dog object  
  
        myAnimal.animalSound();  
        myPig.animalSound();  
        myDog.animalSound();  
    }  
}
```

C# Abstraction

Abstract Classes and Methods

Data **abstraction** is the process of hiding certain details and showing only essential information to the user.

Abstraction can be achieved with either **abstract classes** or [interfaces](#) (which you will learn more about in the next chapter).

The **abstract** keyword is used for classes and methods:

- **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
- **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the derived class (inherited from).

C# Interface

Interfaces

Another way to achieve [abstraction](#) in C#, is with interfaces.

An **interface** is a completely "**abstract class**", which can only contain abstract methods and properties (with empty bodies):

Example [Get your own C# Server](#)

```
// interface  
  
interface Animal  
{  
    void animalSound(); // interface method (does not have a body)  
    void run(); // interface method (does not have a body)  
}
```

C# Enums

An **enum** is a special "class" that represents a group of **constants** (unchangeable/read-only variables).

To create an **enum**, use the **enum** keyword (instead of class or interface), and separate the enum items with a comma:

Example [Get your own C# Server](#)

```
enum Level  
{  
    Low,  
    Medium,  
    High  
}
```

You can access **enum** items with the **dot** syntax:

```
Level myVar = Level.Medium;  
Console.WriteLine(myVar);
```

Enum is short for "enumerations", which means "specifically listed".

Enum inside a Class

You can also have an `enum` inside a class:

Example

```
class Program
{
    enum Level
    {
        Low,
        Medium,
        High
    }

    static void Main(string[] args)
    {
        Level myVar = Level.Medium;
        Console.WriteLine(myVar);
    }
}
```

Enum Values

By default, the first item of an enum has the value 0. The second has the value 1, and so on.

To get the integer value from an item, you must [explicitly convert](#) the item to an `int`:

Example

```
enum Months
{
    January,    // 0
    February,   // 1
    March,      // 2
    April,      // 3
    May,        // 4
    June,       // 5
    July        // 6
}

static void Main(string[] args)
{
    int myNum = (int) Months.April;
```



```
Console.WriteLine(myNum);  
}
```

The output will be:

3

You can also assign your own enum values, and the next items will update their numbers accordingly:

Example

```
enum Months  
{  
    January,    // 0  
    February,   // 1  
    March=6,    // 6  
    April,      // 7  
    May,        // 8  
    June,       // 9  
    July        // 10  
}  
  
static void Main(string[] args)  
{  
    int myNum = (int) Months.April;  
    Console.WriteLine(myNum);  
}
```

The output will be:

7

Enum in a Switch Statement

Enums are often used in `switch` statements to check for corresponding values:

Example

```
enum Level
{
    Low,
    Medium,
    High
}

static void Main(string[] args)
{
    Level myVar = Level.Medium;
    switch(myVar)
    {
        case Level.Low:
            Console.WriteLine("Low level");
            break;
        case Level.Medium:
            Console.WriteLine("Medium level");
            break;
        case Level.High:
            Console.WriteLine("High level");
```

```
break;
```

C# Exceptions - Try..Catch

C# Exceptions

When executing C# code, different errors can occur: coding errors made by the programmer, errors due to wrong input, or other unforeseeable things.

When an error occurs, C# will normally stop and generate an error message. The technical term for this is: C# will throw an **exception** (throw an error).

C# provides a structured way to handle exceptions using the **try**, **catch**, **finally**, and **throw** keywords. The `try` block contains code that may generate an exception. If an error occurs, control is transferred to the `catch` block, where the exception is handled. Multiple catch blocks can be used to handle different types of exceptions. The `finally` block contains code that always executes, regardless of whether an exception occurs, and is typically used for cleanup tasks like closing files or database connections.

Common exception types in C# include **NullReferenceException**, which occurs when accessing a null object, **DivideByZeroException**, which occurs when dividing a number by zero, and **IndexOutOfRangeException**, which occurs when accessing an invalid array index.

Developers can also create custom exceptions by inheriting from the base `Exception` class. The `throw` keyword is used to explicitly raise exceptions when an error condition is detected.

Proper exception handling improves program reliability, maintains application stability, and provides meaningful error messages to users or developers. It is an essential concept in building robust and user-friendly applications in C#.